PYTHON MACHINE LEARNING · MACHINE LEARNING FOUNDATIONS · COMPLETE WORKBOOK

Machine Learning Foundations

Your working notebook for all 11 sessions of Machine Learning Foundations. Read the key ideas, study the worked examples, do the guided labs, predict before you run, and check yourself with the self-check quizzes.

11 sessions · 21 lessons | Write-in exercises & self-checks | Name: _____

CONTENTS

S01 **Your ML Toolkit and the Estimator API**

- 0.1 Your ML Toolkit
- 1.1 The scikit-learn Estimator API

S02 **Splits, Leakage and Cross-Validation**

- 1.2 Splits, Baselines and Leakage
- 1.3 Cross-Validation

S03 **Metrics and Preprocessing**

- 1.4 Metrics for Classification and Regression
- 2.1 Impute, Scale and Encode

S04 **Pipelines and Creating Features**

- 2.2 Pipelines
- 2.3 Creating Features

S05 **Feature Selection and Imbalanced Classes**

- 2.4 Feature Selection
- 2.5 Imbalanced Classes

S06 **Linear Models and Regularization**

- 3.1 Linear and Logistic Regression
- 3.2 Regularization and the Bias-Variance Trade-off

S07 **Nearest Neighbors and Naive Bayes**

- 4.1 k-Nearest Neighbors
- 4.2 Naive Bayes

S08 **Support Vector Machines and Decision Trees**

- 4.3 Support Vector Machines
- 5.1 Decision Trees

S09 **Random Forests and Gradient Boosting**

- 5.2 Bagging and Random Forests
- 5.3 Gradient Boosting

S10 **Ensembles and Model Selection**

- 5.4 Voting and Stacking
- 6.1 Model Selection and Learning Curves

S11 **Hyperparameter Tuning and Final Project Studio**

- 6.2 Hyperparameter Tuning

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 1: Your ML Toolkit and the Estimator API

Your workbook for Session 1: the toolkit check, a diagnostic, the key ideas of Lessons 0.1 and 1.1, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 0.1–1.1 | Session 1 · 3 hours | Name: _____

SESSION GOALS

- Prove your machine is ready: the environment check ends with READY, and you know the five steps that start every work session.
- Read a table the way a data scientist does: what one row is, which column is the target, where the gaps are and why, and what "knowing nothing" already scores.
- Tell supervised from unsupervised learning and classification from regression, on Pinecrest's real requests.
- Train your first models with the four names every scikit-learn model shares: create, `fit`, `predict`, `score`, plus `fit` and `transform` for transformers.
- Judge a model against a baseline, and see why ranking customers by probability can matter more than accuracy.

START HERE: IS YOUR TOOLKIT READY?

ENVIRONMENT CHECK

- ☐ I can open a terminal and get to my project folder: `cd pinecrest-ml`
- ☐ My prompt starts with `(.venv)` after I activate. My activate command is:
  _____
- ☐ `python labs/module-01/l0_1-orientation/check_environment.py` ends with `READY for Python Machine Learning`
- ☐ If it said NOT READY, I copied the numbered fix list here and will do it before Session 2:

The five steps for every time you sit down: **1** open a terminal · **2** `cd pinecrest-ml` · **3** activate, look for `(.venv)` · **4** start JupyterLab or VS Code from this folder · **5** in a notebook, check that the kernel is the `.venv`

one.

DIAGNOSTIC: PREDICT, THEN CHECK

Six questions on the Python and pandas this course builds on. Circle your answer before the class reveals it. Nobody collects this. Missed one? The skill and what to review: **D1** slicing (Intermediate Python: lists and slicing) · **D2** dictionaries (Intermediate Python: dictionaries and .get()) · **D3** functions and default arguments (Intermediate Python: functions) · **D4** boolean filtering (Python Data Science Lesson 2.1) · **D5** groupby (Python Data Science Lesson 2.4) · **D6** missing values (Python Data Science Lesson 2.3). For more practice at home, fill in `labs/module-01/l0_1-orientation/diagnostic_starter.py`: one small function per question, with a checker.

D1

What does this print?

```
items = [5, 6, 7, 8, 9]
print(items[-3:], items[:2])
```

A) `[7, 8, 9] [5, 6]`

B) `[8, 9] [5, 6, 7]`

C) `[7, 8, 9] [5, 6, 7]`

D) `[9, 8, 7] [5, 6]`

D2

What does this print?

```
counts = {}
for store in ["Bend", "Boise", "Bend"]:
    counts[store] = counts.get(store, 0) + 1
print(counts)
```

A) `{'Bend': 1, 'Boise': 1}`

B) `{'Bend': 2}`

C) `{'Bend': 2, 'Boise': 1}`

D) A `KeyError: 'Bend'`

D3

What does this print?

```
def discounted(price, rate=0.10):
    return round(price * (1 - rate), 2)
print(discounted(80), discounted(80, rate=0.25))
```

A) `72.0 72.0`

B) `72.0 60.0`

C) `8.0 20.0`

D) A `TypeError`: rate is given twice

D4

What does this print?

```
import pandas as pd
df = pd.DataFrame({"app_user": [1, 0, 1], "spend": [100.0, 50.0, 300.0]})
print(df.loc[df["app_user"] == 1, "spend"].mean())
```

A) `150.0`

B) `400.0`

C) `2`

D) `200.0`

D5

What does this print?

```
import pandas as pd
df = pd.DataFrame({"tier": ["Gold", "Basic", "Basic"], "churned": [0, 1, 0]})
print(df.groupby("tier")["churned"].mean().to_dict())
```

A) `{'Basic': 0.5, 'Gold': 0.0}`

B) `{'Gold': 0.0, 'Basic': 1.0}`

C) `{'Basic': 1, 'Gold': 0}`

D) `{'Basic': 0.333, 'Gold': 0.0}`

D6

What does this print?

```
import pandas as pd
s = pd.Series([1.0, None, 3.0, None])
print(s.isna().sum(), s.mean())
```

A) `2 1.0`

B) `0 nan`

C) `2 2.0`

D) `2 nan`

LESSONS

0.1 Your ML Toolkit

BY THE END YOU CAN

- Create a project folder with a virtual environment, install the course packages, and confirm that scikit-learn and PyTorch import
- Choose between a notebook and a script for a task, and run code in both
- Describe the machine-learning workflow (frame, prepare, train, evaluate, explain, deploy) in your own words
- Tell supervised from unsupervised learning and classification from regression, and name the features and target of a table
- Load the seven Pinecrest data files and summarize each one's size, gaps and target

KEY IDEAS

- **A workshop of your own.** Python is the building, the virtual environment is your workshop, the packages are the tools, the data folder is the raw material. A model is a tool you make by showing it examples.
- **Activation is per terminal window.** No `(.venv)` in the prompt means the computer-wide Python: `ModuleNotFoundError`. `python -m pip` installs into the Python you are running.
- **Notebook to explore, script to keep.** Running notebook cells out of order is worse in machine learning: a model trained on an old split and graded on a new one gives a meaningless score. Restart and run all when in doubt.
- **The workflow:** frame → prepare → train → evaluate → explain → deploy, with a loop from evaluate back to prepare. Most real improvement happens in that loop.
- **Features and a target.** The target is the column you predict (`churned`); features are the columns the model may use. Supervised learning has a target column; unsupervised learning (finding groups) does not.
- **Classification or regression?** A category target (churned, fraud) means classification; a quantity (units sold, dollars) means regression. Ask: would a value halfway between two others make sense?
- **Rare classes make accuracy easy to fake.** 16.5% of customers churn and 0.88% of payments are fraud, so "nobody churns" is 83.5% accurate and "nothing is fraud" 99.12%.
- **Ask why a value is missing.** A gap can be an unfilled form, a signal in itself, a normal day, or a column written after the outcome (a leak).

WORKED EXAMPLE: WHAT DOES A ZERO MEAN?

Pinecrest's store managers keep asking about daily sales. Before anyone predicts anything, look at what one row is and what the zeros mean.

```
import pandas as pd
sales = pd.read_csv("data/daily_store_sales.csv", parse_dates=["date"])
print(sales.shape)
print(sales.head(3))
```

```
(4384, 6)
        date      store    sales  transactions  promo event
0 2023-01-01  Asheville  4838.69            69      0   NaN
1 2023-01-01       Bend  4533.55            65      0   NaN
2 2023-01-01      Boise  6026.92            80      0   NaN
```

One row is one store on one day: 1,096 days × 4 stores = 4,384 rows. The target a forecaster would predict is `sales`, a quantity: regression.

```
print(sales["event"].value_counts())
print("days with sales of 0:", (sales["sales"] == 0).sum())
```

```
event
Holiday rush         72
Memorial Day sale    48
Labor Day sale       48
Black Friday         48
Closed               24
Name: count, dtype: int64
days with sales of 0: 24
```

Exactly 24 days have sales of 0, and exactly 24 rows say `Closed` (Thanksgiving and Christmas, three years, four stores). These zeros are real, not missing: a model must expect them. And most rows have no `event` at all: that gap simply means "an ordinary day". Always ask what a value, or its absence, means.

GUIDED LAB

- Open `labs/module-01/l0_1-orientation/missing_values_starter.py`. Run it once as it is: it prints the gap counts and four TODO reminders.
- TODO 1 and 2 (together): the share of missing `age_group` for online and store shoppers; the churn rate when `email_opens_90d` is missing and when it is not.
- TODO 3 to 5 (in pairs): a crosstab of "has a cancellation reason" against `churned`; the unrated interactions; one line of verdict per column: why is it missing, and may a model use it?
- Done when your numbers match the "Expected when done" line the starter prints.

- Finished early? `frame_targets_starter.py` in the same folder: frame three new requests from Pinecrest teams.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code. A wrong prediction you can explain afterward is worth more than a right guess.

P1

Predict the output, then run it. (Two of ten flags are 1.)

```
import pandas as pd
flags = pd.Series([1, 0, 0, 0, 0, 0, 0, 1, 0, 0])
print(flags.mean(), round(1 - flags.mean(), 1))
```

P2

Predict the output, then run it. (Two gaps out of four values.)

```
import pandas as pd
s = pd.Series([3.0, None, 5.0, None])
print(s.isna().sum(), s.mean())
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The data card (lesson page, "Project: A Data Card for the Course Files"). In class: Steps 1 and 2. At home: Steps 3 to 6.

- ☐ Step 1: seven tables in a dictionary. Check: `7 tables loaded`, `106,746 rows in total`
- ☐ Step 2: the gaps per table (three tables have gaps)
- ☐ Step 3: a target and a problem kind for every file, my own judgment
- ☐ Step 4: churn 16.53% of rows, fraud 0.88%

- ☐ Step 5: `data_overview.csv` with seven rows
- ☐ Step 6: `journal.md` exists

SELF-CHECK

Q1

In JupyterLab, `import sklearn` fails with `ModuleNotFoundError`, yet in your activated terminal `python -c "import sklearn"` works. What is the most likely cause?

A) scikit-learn is only partly installed: reinstall it

B) The import name is wrong: use `import scikit_learn`

C) The notebook's kernel is a different Python from the one in `.venv`

D) JupyterLab cannot run scikit-learn; use a script instead

Q2

Which of these requests is a regression problem?

A) Will this customer stop buying in the next six months?

B) Which customers shop in similar ways? (No column says.)

C) Is this product review positive or negative?

D) How many dollars will the Boise store take tomorrow?

Q3

What does this print?

```
import pandas as pd
s = pd.Series([0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0])
print(round(s.mean(), 2), s.sum())
```

Q4

Which column of `customers_churn.csv` must never be a feature for predicting churn?

A) `email_opens_90d`, because it has 537 missing values

B) `cancellation_reason`, because it is filled in only after a customer has left

C) `home_store`, because it is text

D) `app_user`, because it only holds 0 and 1

Q5

Your model scores 0.840 accuracy on held-out customers, and a model that always predicts "stays" scores 0.835. Which workflow step are you in, and where does the loop send you if this is not good enough?

A) Evaluate; back to Prepare

B) Train; on to Explain

C) Explain; back to Frame

D) Deploy; on to Monitor

Q6

What does this print?

```
import pandas as pd
customers = pd.read_csv("data/customers_churn.csv")
print(customers["signup_date"].dtype)
```

JOURNAL

Pick one problem from your own work or life that a model might help with. Frame it as the workflow's first step would: what exactly is predicted, what would the features and the target be, is it classification or regression, and what does a wrong prediction cost?

1.1 The scikit-learn Estimator API

BY THE END YOU CAN

- Turn a table into a feature matrix `X` and a target `y`, and hold out a test set with `train_test_split`
- Use the estimator methods `fit`, `predict`, `predict_proba` and `score`, and read learned attributes such as `coef_`
- Use a transformer with `fit` and `transform`, fitting it on the training data only
- Measure a `DummyClassifier` baseline and judge whether a real model beats it
- Swap one model for another without changing the rest of your code

KEY IDEAS

- **The new hire and the case files.** `fit` = study the old files; `predict` = answer fresh files with the ending torn off; `score` = the manager's grade. Grade only with files the new hire has never seen.
- **X and y.** `X` is a table (two dimensions, even with one feature: double brackets); `y` is one column. Today only numeric columns without gaps.
- `train_test_split(X, y, test_size=0.25, stratify=y, random_state=42)` keeps a quarter for testing, the same churn share on both sides, and the same rows every time. Results come back as X parts, then y parts.
- **Every estimator:** create with settings (`get_params()`), `fit` (returns the estimator), then `predict`, `predict_proba` or `score` (accuracy for classifiers, R^2 for regressors). Names ending in `_` were learned during `fit`.
- **Transformers** use `fit` and `transform`. Fit the scaler on the training rows only, then transform both sets. `set_output(transform="pandas")` keeps the column names.
- **A baseline first.** `DummyClassifier(strategy="most_frequent")` scores 0.835 on churn while finding nobody. A model earns its place only by beating it.
- **Probabilities rank.** `predict_proba(X)[:, 1]` is the churn probability (column order = `classes_`). Sorted into deciles, the riskiest tenth churned at 57%: a model barely better on accuracy can be very useful.
- **Swap models, keep the code.** Because every model shares the API, one loop can compare a baseline, logistic regression, a tree and a forest.

WORKED EXAMPLE: ONE FEATURE, FOUR NEW CUSTOMERS

A model on ONE feature, so you can see exactly what `fit` learned. The question: how does the time since a customer's last order change the predicted churn risk?

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
customers = pd.read_csv("data/customers_churn.csv")
X = customers[["days_since_last_order"]]          # double brackets: a one-column
y = customers["churned"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, stratify
tree = DecisionTreeClassifier(max_depth=2, random_state=42).fit(X_train, y_train)
print(f"test accuracy: {tree.score(X_test, y_test):.3f}")
```

```
test accuracy: 0.842
```

Now score four customers who are in no file: 10, 120, 200 and 330 days since their last order.

```
new = pd.DataFrame({"days_since_last_order": [10, 120, 200, 330]})
print(tree.predict(new))
print(tree.predict_proba(new)[:, 1].round(3))
```

```
[0 0 0 1]
[0.097 0.097 0.494 0.595]
```

A depth-2 tree splits the days into four bands and predicts the churn share of each band. Customers who ordered within the last four months or so are at about 10% risk; at 200 days the risk is 49%, and at 330 days 60%. Notice the third customer: a probability of 0.494 is just under 0.5, so `predict` says 0. That cut-off is a choice, and Lesson 1.4 is about choosing it well.

GUIDED LAB

- **Lab A:** `labs/module-01/l1_1-estimator-api/fraud_baseline_starter.py`. The same moves as the demo on card fraud: split 70/30 with `stratify`, a baseline, a scaled logistic regression, then the 100 riskiest payments. Write your prediction for the baseline accuracy in the TODO 2 comment before you run it.
- **Lab B (pairs):** `labs/module-01/l1_1-estimator-api/new_customers_starter.py`. Score three customers who are in no file. One of you types, the other predicts each answer out loud; swap at TODO 4.
- Done when your numbers match the "Expected when done" lines.
- Finished early? `debug_estimator.py`: three bugs that stop it, and one that does not.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code. A wrong prediction you can explain afterward is worth more than a right guess.

P3

Predict the output, then run it. (Three training rows, labels 1, 1, 0.)

```
from sklearn.dummy import DummyClassifier
d = DummyClassifier(strategy="most_frequent").fit([[0], [1], [2]], [1, 1, 0])
print(d.predict([[5], [9]]))
```

P4

Predict the output, then run it. (The scaler learned a minimum and a maximum.)

```
from sklearn.preprocessing import MinMaxScaler
mm = MinMaxScaler().fit([[10], [20], [30]])
print(mm.transform([[20], [40]]).ravel())
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The churn-risk shortlist (lesson page, "Project: The Churn-Risk Shortlist"). In class: Steps 1 to 3. At home: Steps 4 to 6.

- ☐ Step 1: the split keeps the IDs. Check: `4500 1500 ['C102112', 'C103063', 'C101159']`
- ☐ Step 2: baseline 0.835; 8.3 churners expected in a random list of 50
- ☐ Step 3: logistic regression 0.848, random forest 0.873
- ☐ Step 4: the 50 highest probabilities for each model
- ☐ Step 5: how many on each list really churned (compare with 8.3)
- ☐ Step 6: `churn_shortlist.csv`: IDs and probabilities only, 50 rows

SELF-CHECK

Q7

What happens when this runs?

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
X = pd.DataFrame({"days": [10, 200, 30, 300]}); y = [0, 1, 0, 1]
model = LogisticRegression()
model.fit(X, y)
model = LogisticRegression()
print(model.predict(X))
```

A) It prints `[0 1 0 1]`

B) It prints `[0 0 0 0]`: a new model knows nothing

C) `ValueError`: the features must be scaled first

D) `NotFittedError`: the last assignment replaced the fitted model with a fresh one

Q8

What does this print?

```
from sklearn.preprocessing import StandardScaler
s = StandardScaler().fit([[2], [4], [6]])
print(s.mean_, s.transform([[4], [8]]).ravel().round(3))
```

Q9

A fitted churn model has `classes_` equal to `[0 1]`. What is `model.predict_proba(X_test)[:, 1]`?

A) Each test customer's probability of churning

B) Each test customer's predicted label, 0 or 1

C) Each test customer's probability of staying

D) Both probabilities for the second test customer

Q10

Why does the scaler learn its means and standard deviations from the training rows only?

A) Fitting on fewer rows is faster

B) StandardScaler cannot fit a table that has the target in it

C) The test rows stand in for future customers, whose numbers could not have been seen when the model was built

D) `transform` raises an error on rows the scaler was fitted on

Q11

What happens when this runs?

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
customers = pd.read_csv("data/customers_churn.csv")
LogisticRegression().fit(customers["days_since_last_order"], customers["churne
```

A) It trains a one-feature model

B) `ValueError`: X must be a table; `customers[["days_since_last_order"]]` fixes it

C) `KeyError`: the column does not exist

D) `ConvergenceWarning`: the feature is not scaled

Q12

What does this print?

```
from sklearn.linear_model import LogisticRegression
print(sorted(LogisticRegression().get_params())[:3])
```

JOURNAL

Explain the new hire and the case files in your own words, using fit, predict and score. Then write down the two numbers from your project that tell the most honest story about your model: the baseline accuracy, and how many of your 50 shortlisted customers really churned. Why are those two better together than either alone?

EXIT TICKET

Three questions, no notes. Write your answers, hand them in or keep them: they tell you and your instructor what to revisit.

E1

Your prompt does not start with `(.venv)`. What will probably go wrong, and what do you type to fix it?

E2

Name the three steps every scikit-learn model follows, and the two a transformer uses instead of predicting. Which data does the transformer learn from?

E3

A churn model scores 0.848 accuracy; always predicting "stays" scores 0.835. Is the model useful? What would you check?

NOTES

HOMEWORK CHECKLIST

- ☐ Finish the data card (Steps 3 to 6) and the churn-risk shortlist (Steps 4 to 6)
- ☐ Lesson 0.1 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Lesson 1.1 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Both Quick Quizzes on the lesson pages
- ☐ Both journal entries in `journal.md`
- ☐ 1.1 "Before the Next Lesson": rerun the project with `random_state=7`; predict, then check, what `DummyClassifier(strategy="constant", constant=1)` scores
- ☐ If my check said NOT READY: follow its list and run it again
- ☐ Optional: `diagnostic_starter.py` (if I missed two or more diagnostic questions) and `frame_targets_starter.py`
- ☐ Bring to Session 2: a READY laptop, `pinecrest-ml` with `data/` and `labs/`, my 1.1 project notebook, this workbook and my journal

GLOSSARY

TERM	MEANING
virtual environment	A project's own Python and packages (`.venv`), switched on per terminal with the activate command
kernel	The Python process a notebook runs on; it must be the `.venv` one
sample (row)	One example, such as one customer
feature	A column the model may use as input; together they form `X`
target (label)	The column to predict, `y`
supervised / unsupervised	Learning from rows with the right answer / finding structure without one
classification / regression	Predicting a category / predicting a quantity
baseline	What a model that learns nothing scores; every model is compared with it
estimator	Any scikit-learn object with `fit` ; models and transformers are estimators
transformer	An estimator that changes data with `fit` and `transform` (for example `StandardScaler`)
hyperparameter	A setting chosen before fitting, passed to the constructor (`max_depth=4`)
learned attribute	Something `fit` computed, named with a trailing underscore (`coef_` , `mean_`)
target leakage	A feature that would not be known at prediction time, such as `cancellation_reason`
decile	One tenth of the rows after sorting, here by predicted risk

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 2: Splits, Leakage and Cross-Validation

Your workbook for Session 2: a warm-up from Session 1, the key ideas of Lessons 1.2 and 1.3, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 1.2–1.3 | Session 2 · 3 hours | Name: _____

SESSION GOALS

- Measure how much one test score wobbles, and stop trusting differences smaller than the wobble.
- Give train, validation and test data one job each, and open the test set once.
- Split by time or by group when rows are related, and beat the strongest free baseline, not the weakest.
- Catch a leak with a single-feature audit and the question "known at prediction time?".
- Cross-validate a pipeline on shuffled, stratified folds, compare models fold by fold, and choose folds that match the question.

WARM-UP: RETRIEVAL FROM SESSION 1

Five quick questions on Lessons 0.1 and 1.1. Write your answer first, then compare with a neighbor.

W1

On the churn table, what does always predicting "stays" score? And "never fraud" on the payments?

W2

Which column of `customers_churn.csv` must never be a feature, and why?

W3

`train_test_split(X, y, ...)` returns four things. In which order?

W4

Why is the scaler fitted on the training rows only, and then used to transform both sets?

W5

Homework: what did `DummyClassifier(strategy="constant", constant=1)` score on the 1.1 test set?

LESSONS

1.2 Splits, Baselines and Leakage

BY THE END YOU CAN

- Measure how much a test score moves from one random split to another, and explain why stratifying helps
- Make a train / validation / test split and use each part for its one job
- Split by time and by group when rows are related, and show how much a careless random split flatters a model
- Pick a baseline that fits the problem (most frequent class, mean, yesterday, same day last week)
- Detect target leakage and train-test contamination, and remove them

KEY IDEAS

- **The dress rehearsal rule.** Evaluate the model under the same conditions it will face in real use. Every way of breaking the rule makes the score look better, which is why nobody notices.
- **One split is one sample.** Over 30 seeds the same model scored 0.836 to 0.866 (standard deviation about 0.007). Differences smaller than the wobble mean nothing.
- **Stratify classification splits on the target** (`stratify=y`), so the class share and with it the baseline stay fixed. Never stratify on a feature or a number.
- **Train fits, validation chooses, test grades once.** Call `train_test_split` twice for 60/20/20 (`test_size=0.20`, then `0.25` of the rest). After the one look at the test set, no more choices.
- **Time-ordered data needs a time split** (train on the past, test on the future): on daily sales a random split claimed $613 a day, the truth was $854.
- **Related rows need a group split** when the model will meet new groups: `GroupShuffleSplit(..., groups=...)`. New products: 17.84 units off instead of 10.23.
- **Beat the strongest free baseline:** most frequent class, the mean, yesterday, the same day last week ($1,072 for daily sales).
- **Leakage:** target leakage (recorded after the outcome), train-test contamination (a step fitted before the split), time leakage (the future helps), group leakage. Catch it with a single-feature audit and "known at prediction time?".

WORKED EXAMPLE: THE SAME CUSTOMER ON BOTH SIDES?

Pinecrest's recommendation team wants to know how well a model will do for customers it has never seen. `interactions.csv` has one row per customer and product, so most customers have many rows. Does a

random split give the model strangers to predict?

```
import pandas as pd
from sklearn.model_selection import train_test_split, GroupShuffleSplit
inter = pd.read_csv("data/interactions.csv")
print(inter.shape, inter["customer_id"].nunique(), "customers")
row_train, row_test = train_test_split(inter, test_size=0.25, random_state=42)
shared = set(row_train["customer_id"]) & set(row_test["customer_id"])
print(f"random rows: {row_test['customer_id'].nunique()} test customers, {len(shar
```

```
(32178, 5) 3000 customers
random rows: 2649 test customers, 2641 also in training
```

Almost every test customer (2,641 of 2,649) also has rows in training. A model graded this way is graded on customers it already knows: an easier exam than "new customers". Now split by customer instead.

```
gss = GroupShuffleSplit(n_splits=1, test_size=0.25, random_state=42)
train_idx, test_idx = next(gss.split(inter, groups=inter["customer_id"]))
g_train, g_test = inter.iloc[train_idx], inter.iloc[test_idx]
shared = set(g_train["customer_id"]) & set(g_test["customer_id"])
print(f"group split: {len(g_test):,} test rows, {g_test['customer_id'].nunique()}
      f"{len(shared)} also in training")
```

```
group split: 7,999 test rows, 750 test customers, 0 also in training
```

A quarter of the customers (750) and all their rows go to the test side; none of them appears in training. Which split is right depends on the question: "recommend more products to our existing customers" is a row (or time) split; "how well will this work for new sign-ups?" is a group split. Choosing the split is choosing the question.

GUIDED LAB

- Open `labs/module-02/l1_2-splits-and-leakage/demand_forecast_starter.py`. Run it once as it is: it prints the table's size and five TODO reminders.
- TODO 1 (together): the time split, 2024 for training and 2025 for testing. Predict first: how many weeks on each side?
- TODO 2 and 3 (pairs): a random split of the same size, and the same forest graded both ways. Write your prediction in the comment before you run TODO 3.
- TODO 4 and 5: three free baselines (training mean, each product's 2024 average, the same week last year) and one sorted table: does the forest beat the strongest?
- Done when your numbers match the "Expected when done" lines the starter prints.

- Finished early? `fraud_validation_starter.py` (a validation set chooses a fraud model) or `debug_leaky_split.py` (two loud bugs, two silent ones).

PREDICT, THEN RUN

Write your prediction on the line first, then run the code. A wrong prediction you can explain afterward is worth more than a right guess.

P1

Predict the output, then run it. (An integer test_size.)

```
from sklearn.model_selection import train_test_split
a, b = train_test_split(list(range(20)), test_size=5, random_state=0)
print(len(a), len(b))
```

P2

Predict the output, then run it. (16 zeros and 4 ones, stratified.)

```
from sklearn.model_selection import train_test_split
y = [0] * 16 + [1] * 4
y_tr, y_te = train_test_split(y, test_size=0.25, stratify=y, random_state=0)
print(len(y_te), sum(y_te))
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

An honest churn evaluation (lesson page, "Project: An Honest Churn Evaluation"). In class: Steps 1 to 3. At home: Steps 4 to 6.

- ☐ Step 1: three parts. Check: `3600 1200 1200`; from now on `test` is sealed
- ☐ Step 2: the leak audit on training and validation rows. Check: `suspects: ['has_reason']`
- ☐ Step 3: drop the leak; baseline validation accuracy 0.835

- ☐ Step 4: choose on validation (logistic regression 0.850, tree 0.864, forest 0.870)
- ☐ Step 5: open the test set once: forest 0.866, baseline 0.835
- ☐ Step 6: `evaluation_report.txt` built from my variables

SELF-CHECK

Q1

Pinecrest wants to predict how a product it has never sold will sell, from `weekly_demand.csv`. Which evaluation matches how the model will be used?

A) A random 75/25 split of the rows with `stratify=demand["units_sold"]`

B) A time split: 2024 for training, 2025 for testing

C) A group split, `GroupShuffleSplit(..., groups=demand["product_id"])`, so that no test product appears in training

D) Cross-validate on all the rows, then confirm on the same rows

Q2

What does this print?

```
from sklearn.model_selection import train_test_split
rows = list(range(1000))
rest, test = train_test_split(rows, test_size=0.20, random_state=42)
train, val = train_test_split(rest, test_size=0.25, random_state=42)
print(len(train), len(val), len(test))
```

Q3

You forecast tomorrow's sales for each store. Which candidate feature leaks the future?

A) Yesterday's sales, `shift(1)` within each store

B) A 7-day average built with `rolling(7, center=True)`

C) Sales on the same day last week, `shift(7)`

D) The day of the week

Q4

What happens when this runs?

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
customers = pd.read_csv("data/customers_churn.csv")
X, y = customers[["tenure_months", "orders_12m"]], customers["churned"]
X_train, y_train, X_test, y_test = train_test_split(X, y, test_size=0.25, rand
LogisticRegression().fit(X_train, y_train)
```

A) It fits a model on 4,500 customers

B) `KeyError`: `'churned'`

C) `NotFittedError`: the model was never fitted

D) `ValueError`: `y` should be one-dimensional; the second result is the 1,500 test rows of `X`

Q5

Depths 3 and 4 tie on validation (0.864). You refit depth 3 on training plus validation rows and it scores 0.858 on the test set. What now?

A) Report 0.858 and keep depth 3: the test set has spoken once

B) Report 0.864: it is the best number you measured

C) Try depth 4 on the test set too and report the better of the two

D) Report the average of 0.864 and 0.858

Q6

What does this print? (The "yesterday" forecast's mean absolute error.)

```
import pandas as pd
sales = pd.Series([100, 120, 90, 130])
yesterday = sales.shift(1)
print((sales - yesterday).abs().mean())
```

JOURNAL

Explain the dress rehearsal rule in your own words. Then take the problem you framed in Lesson 0.1's journal entry: should it be split at random, by time or by group; which baseline would you compare with; and which column in your data would be most tempted to leak?

1.3 Cross-Validation

BY THE END YOU CAN

- Explain why one train/validation split gives a noisy score, and how k-fold cross-validation averages that noise away
- Run `cross_val_score` and `cross_validate` with a stratified, shuffled splitter, and report a mean with its spread
- Keep preprocessing inside the folds by cross-validating a `make_pipeline` instead of pre-scaled data
- Compare two models fold by fold on identical folds, and get out-of-fold predictions with `cross_val_predict`
- Choose `TimeSeriesSplit` or `GroupKFold` when rows are related by time or by group

KEY IDEAS

- **One exam or five quizzes.** K-fold cross-validation trains k times, validates every training row exactly once, and gives k scores: report the mean plus or minus the spread.
- **It scores a recipe, not a model.** `cross_val_score` fits fresh copies and throws them away; afterward fit the chosen recipe once on all the training rows.
- **Build the splitter yourself:** `StratifiedKFold(5, shuffle=True, random_state=42)` for classification, `KFold(5, shuffle=True, random_state=42)` for regression. `cv=5` does not shuffle.
- **Preprocessing goes inside the folds:** cross-validate `make_pipeline(StandardScaler(), LogisticRegression())`, never pre-scaled data. Selecting features outside the folds turned pure noise into 0.825 accuracy.
- `cross_validate` gives several metrics and, with `return_train_score=True`, the training score: a big train-validation gap means overfitting.
- **Compare on identical folds and look at paired differences.** A lead smaller than the fold-to-fold spread is a tie (12 of 15 folds, +0.004: a tie). Then prefer the simpler model.
- `cross_val_predict` gives every training row an honest out-of-fold prediction, for charts and thresholds.
- **Match the folds to the question: what will be new to the model?** New customers: shuffled stratified folds. Next period: time-ordered folds. A new product or store: `GroupKFold`.

WORKED EXAMPLE: RECENCY ALONE, FIVE QUIZZES

How much does recency alone tell you about churn, honestly? Cross-validate a two-level tree on ONE feature against the know-nothing baseline, with training scores next to the validation scores.

```
import pandas as pd
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_valic
from sklearn.tree import DecisionTreeClassifier
from sklearn.dummy import DummyClassifier
customers = pd.read_csv("data/customers_churn.csv")
X_train, X_test, y_train, y_test = train_test_split(
    customers[["days_since_last_order"]], customers["churned"],
    test_size=0.25, stratify=customers["churned"], random_state=42)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
for name, model in [("baseline", DummyClassifier(strategy="prior")),
                    ("recency tree", DecisionTreeClassifier(max_depth=2, random_st
    r = cross_validate(model, X_train, y_train, cv=cv, scoring="roc_auc", return_t
    print(f"{name:12s} AUC {r['test_score'].mean():.3f} +/- {r['test_score'].std()
          f"  (train {r['train_score'].mean():.3f})")
```

```
baseline     AUC 0.500 +/- 0.000  (train 0.500)
recency tree AUC 0.703 +/- 0.023  (train 0.709)
```

Ranking AUC 0.703 against 0.500 for the baseline: recency alone carries real signal. Train and validation are close (0.709 vs 0.703), so the small tree is not memorizing. Now look inside the five rounds: cross-validation fits five different models.

```
for fold, (tr, va) in enumerate(cv.split(X_train, y_train), start=1):
    tree = DecisionTreeClassifier(max_depth=2, random_state=42).fit(X_train.iloc[t
    print(f"fold {fold}: first question 'days_since_last_order <= {tree.tree_.thre
```

```
fold 1: first question 'days_since_last_order <= 172.5?'
fold 2: first question 'days_since_last_order <= 172.5?'
fold 3: first question 'days_since_last_order <= 164.5?'
fold 4: first question 'days_since_last_order <= 172.0?'
fold 5: first question 'days_since_last_order <= 158.5?'
```

Five fold models, five slightly different first questions, all near 160 to 170 days (the churn risk jumps for customers who have not ordered in about five months). That is why cross-validation scores a **recipe** ("a depth-2 tree on recency"), not one model: afterward you fit the recipe once on all the training rows.

GUIDED LAB

- Open `labs/module-02/l1_3-cross-validation/fraud_folds_starter.py`. June's payments are locked away; everything happens on January to May.
- TODO 1 (together): frauds in each validation fold for unshuffled `KFold` and for `StratifiedKFold`. Predict which one gives equal counts.

- TODO 2 and 3 (pairs): cross-validate a baseline, a logistic-regression pipeline and a tree with training scores; then compare two of them fold by fold.
- TODO 4 and 5: next-month folds built by hand, then June, once.
- Done when your numbers match the "Expected when done" lines.
- Finished early? `regularization_check_starter.py`: Session 1 said C=0.01 beat C=1.0 on one split. Do 15 paired folds agree? Then `debug_cross_validation.py`.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code. A wrong prediction you can explain afterward is worth more than a right guess.

P3

Predict the output, then run it. (8 zeros and 4 ones, four stratified folds: ones per validation fold.)

```
import numpy as np
from sklearn.model_selection import StratifiedKFold
y = np.array([0] * 8 + [1] * 4)
cv = StratifiedKFold(n_splits=4, shuffle=True, random_state=42)
print([int(y[val].sum()) for _, val in cv.split(np.zeros((12, 1)), y)])
```

P4

Predict the output, then run it. (Ten rows in time order, three time-ordered folds.)

```
import numpy as np
from sklearn.model_selection import TimeSeriesSplit
for train, val in TimeSeriesSplit(n_splits=3).split(np.zeros((10, 1))):
    print(len(train), val.tolist())
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

A fair model bake-off (lesson page, "Project: A Fair Model Bake-Off"). In class: Steps 1 to 3. At home: Steps 4 to 6. Keep this notebook for Session 3.

- ☐ Step 1: the standard split. Check: `4500 1500`
- ☐ Step 2: four candidates; the scaler inside the logistic-regression pipeline
- ☐ Step 3: 15 identical folds, two metrics. Check: random forest on top, ROC AUC 0.867 ± 0.018; baseline 0.500
- ☐ Step 4: the lead fold by fold (15 of 15, mean +0.033)
- ☐ Step 5: the test set once (forest 0.865)
- ☐ Step 6: `cv_leaderboard.csv` and a two-line recommendation

SELF-CHECK

Q7

What does this print?

```
from sklearn.model_selection import RepeatedStratifiedKFold
rcv = RepeatedStratifiedKFold(n_splits=4, n_repeats=2, random_state=42)
print(rcv.get_n_splits())
```

Q8

What happens when this runs?

```
from sklearn.model_selection import KFold
cv = KFold(n_splits=5, random_state=42)
```

A) It makes five shuffled folds with seed 42

B) `ValueError`: setting a `random_state` has no effect since `shuffle` is False

C) It makes five unshuffled folds and silently ignores the seed

D) `TypeError`: `KFold` has no `random_state` argument

Q9

A colleague fills the gaps in `email_opens_90d` with the median of all 4,500 training rows, then runs `cross_val_score` on the filled table. What is wrong?

A) Nothing: the test set was never used

B) `cross_val_score` cannot handle a column that once had gaps

C) The median should come from all 6,000 rows to be more accurate

D) Each validation fold helped compute the median that filled it: put the filling step inside a pipeline so it is refitted on the training folds of every round

Q10

`interactions.csv` has many rows per customer. You want to know how well a model will predict ratings for customers it has never seen. Which splitter?

A) `StratifiedKFold(5, shuffle=True, random_state=42)` on the rating

B) `TimeSeriesSplit(n_splits=5)` on the rows in file order

C) `GroupKFold(n_splits=5)` with `groups=customer_id`

D) `KFold(5)` without shuffling

Q11

What does this print?

```
from sklearn.dummy import DummyRegressor
from sklearn.model_selection import KFold, cross_val_score
X = [[0]] * 6
y = [1, 2, 3, 4, 5, 6]
print(cross_val_score(DummyRegressor(), X, y, cv=KFold(3), scoring="neg_mean_a
```

Q12

On 15 paired folds the forest beats the depth-4 tree in all 15 (CV AUC 0.867 +/- 0.018). Fitted on all training rows, it then scores 0.865 on the test set. What do you conclude?

A) The estimate held: report 0.865 and keep the forest

B) The forest got worse on the test set; switch to the tree

C) Cross-validation was optimistic; rerun it with the test rows included

D) Report 0.867, because it is based on more folds

◆ JOURNAL

Explain "one exam or five quizzes" to a colleague who has never heard of cross-validation. Then write the question you would ask before choosing a splitter ("what will be new to the model?") and answer it for a project you care about: shuffled folds, time-ordered folds or group folds?

EXIT TICKET

Three questions, no notes. Write your answers, hand them in or keep them: they tell you and your instructor what to revisit.

E1

Name the three parts of a three-way split and the one job of each. Which part is opened only once?

E2

A sales forecast scores MAE $613 on a random split and $854 when trained on 2023 and 2024 and tested on 2025. Which number do you report, and why is the other one lower?

E3

Why do you cross-validate `make_pipeline(StandardScaler(), LogisticRegression())` instead of scaling `X_train` first and cross-validating the model?

NOTES

HOMEWORK CHECKLIST

- ☐ Finish the honest churn evaluation (Steps 4 to 6) and the fair bake-off (Steps 4 to 6)
- ☐ Lesson 1.2 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Lesson 1.3 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Both Quick Quizzes on the lesson pages
- ☐ Both journal entries in `journal.md`
- ☐ 1.2 "Before the Next Lesson": leak hunt in `data/README.md`; rerun the 1.2 project with `random_state=7`
- ☐ 1.3 "Before the Next Lesson": rerun the bake-off with `scoring=["accuracy", "roc_auc", "f1"]` and note the baseline's F1; one dataset of my own with related rows
- ☐ Optional: `fraud_validation_starter.py`, `regularization_check_starter.py`, the two `debug_*.py` handouts
- ☐ Bring to Session 3: a READY laptop, `pinecrest-ml` with `data/` and `labs/`, my 1.3 bake-off notebook and `cv_leaderboard.csv`, this workbook and my journal

GLOSSARY

TERM	MEANING
wobble (split noise)	How much a score moves when only the random split changes
stratify	Keep the class share the same in every part of a split (`stratify=y`)
validation set	Data used to compare candidates and choose one; it may influence your choices
test set	Data locked away until the choice is made, then used once to report the result
time split	Train on earlier rows, test on later ones, as in real use
group split	Every row of the same entity (product, customer, store) on one side
naive / seasonal naive	Forecast baselines: tomorrow = today; next Saturday = last Saturday
MAE	Mean absolute error: the average size of a miss, in the target's units
data leakage	Information during training or testing that the model will not have in real use
target leakage	A feature recorded after, or because of, the outcome (`cancellation_reason`)
train-test contamination	Test or validation rows influence training, e.g. a scaler fitted before the split
k-fold cross-validation	Train k times, each time validating on a different fold; report mean and spread
fold	One of the k equal parts the training rows are cut into
pipeline	Steps chained into one estimator (`make_pipeline`), refitted together in every fold
out-of-fold prediction	A prediction for a row from the fold model that did not train on it (`cross_val_predict`)
ROC AUC	How well a model ranks positives above negatives: 0.5 random, 1.0 perfect (Lesson 1.4)

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 3: Metrics and Preprocessing

Your workbook for Session 3: a warm-up on Session 2, the key ideas of Lessons 1.4 and 2.1, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 1.4–2.1 | Session 3 · 3 hours | Name: _____

SESSION GOALS

- Turn a confusion matrix into precision and recall, and say each in a sentence a manager can act on.
- Move a decision threshold, and choose one from business costs using training data only, never the test set.
- Judge a ranking with ROC AUC and average precision, and know which one to trust when the positives are rare.
- Grade a forecast with MAE, RMSE and R^2 against a naive baseline, and recognize the MAPE trap.
- Prepare a whole table, gaps, units and words included, with imputers, scalers, encoders and one ColumnTransformer learned from the training rows.

WARM-UP: SESSION 2 IN FIVE QUESTIONS

ANSWER ALONE, THEN COMPARE WITH A NEIGHBOR

From Lessons 1.2 and 1.3. Write a sentence or two for each. Your instructor will go through them in the first ten minutes.

W1

Why does every churn split in this course pass `stratify=y`?

W2

In the honest-evaluation project, a depth-2 tree on the single column `has_reason` scored 0.942 on validation against a baseline of 0.835. What do you do, and what do you check first?

W3

Why cross-validate `make_pipeline(StandardScaler(), LogisticRegression())` instead of scaling `X_train` once and cross-validating the model on the scaled table?

W4

What does `cross_val_predict(model, X_train, y_train, cv=cv, method="predict_proba")[:, 1]` give you, and why is it honest?

W5

You reran the bake-off with `scoring=["accuracy", "roc_auc", "f1"]`. What did the baseline score on accuracy and on F1, and why so different?

LESSONS

1.4 Metrics for Classification and Regression

BY THE END YOU CAN

- Read a confusion matrix and compute precision, recall and F1, saying what each means for Pinecrest's retention team
- Move a classifier's decision threshold and explain the precision-recall trade-off it controls
- Use ROC AUC and average precision to judge how well a model ranks, and know when the precision-recall view is the honest one
- Choose a threshold from business costs with out-of-fold probabilities or TunedThresholdClassifierCV, without peeking at the test set
- Report MAE, RMSE and R^2 for a regression against a baseline, and avoid the MAPE trap on zero sales

KEY IDEAS

- **The smoke alarm.** Recall: when there is a fire, does it go off? Precision: when it goes off, is there a fire? The threshold: how sensitive to set it, which depends on what each mistake costs.
- **The confusion matrix** counts TN, FP, FN and TP (rows: the truth; columns: the prediction). `confusion_matrix(y_test, y_pred).ravel()` gives them in that order.
- **Precision = TP / (TP + FP)**, "of those we called, how many were leaving?"; **recall = TP / (TP + FN)**, "of those leaving, how many did we call?"; **F1** is their harmonic mean.
- **The threshold is a decision, not a property of the model.** Lower it: recall up, precision down. `FixedThresholdClassifier` saves a threshold with the model. Never choose it on the test set.
- **ROC AUC** = the chance a random churner outranks a random stayer (0.5 random, 1.0 perfect). **Average precision** summarizes the precision-recall curve; its random baseline is the positive rate (0.165 churn, about 0.009 fraud). For rare positives it is the more honest number.
- **Thresholds from costs.** Profit = value × TP − cost × (TP + FP); break-even probability = cost / value. Choose on out-of-fold probabilities, or with `TunedThresholdClassifierCV(model, scoring=make_scorer(profit), cv=cv)`, then test once.
- **Regression:** MAE (average miss, in units), RMSE (punishes big misses, always at least the MAE), R^2 (against predicting the mean). Always next to a naive baseline. MAPE explodes when true values can be 0.
- **Metric functions take** `(y_true, y_pred)`, AUC needs scores not labels, and in scikit-learn 1.9 they return plain floats (`round(x, 3)`).

WORKED EXAMPLE: ONE THRESHOLD, TWO BUSINESS ANSWERS

A small tree on three columns, so you can follow every number. The question: what does moving the threshold from 0.5 to 0.3 do to the retention team's calls, and to its profit at $15 per offer and $60 per churner reached?

```
import pandas as pd
from sklearn.metrics import confusion_matrix, precision_score, recall_score
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
customers = pd.read_csv("data/customers_churn.csv")
X = customers[["days_since_last_order", "discount_share", "tenure_months"]]
y = customers["churned"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, stratify
tree = DecisionTreeClassifier(max_depth=4, random_state=42).fit(X_train, y_train)
proba = tree.predict_proba(X_test)[:, 1]
for t in [0.5, 0.3]:
    flagged = (proba >= t).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_test, flagged).ravel()
    print(f"threshold {t}: flagged {flagged.sum()}  TP {tp}  FP {fp}  FN {fn}  "
          f"precision {precision_score(y_test, flagged):.3f}  recall {recall_score
```

```
threshold 0.5: flagged 126  TP 87  FP 39  FN 161  precision 0.690  recall 0.351
threshold 0.3: flagged 274  TP 152  FP 122  FN 96  precision 0.555  recall 0.613
```

Read the first line as the care team would: 126 calls, 87 reach real churners, 39 are wasted, and 161 churners are missed. Lowering the threshold to 0.3 more than doubles the calls (274) and finds 152 churners: recall rises from 0.351 to 0.613 while precision falls from 0.690 to 0.555. Now put money on it.

```
for t in [0.5, 0.3]:
    flagged = (proba >= t).astype(int)
    tp = ((flagged == 1) & (y_test == 1)).sum()
    print(f"threshold {t}: profit {60 * tp - 15 * flagged.sum():,} dollars")
```

```
threshold 0.5: profit 3,330 dollars
threshold 0.3: profit 5,010 dollars
```

The lower threshold earns more, because $60 for a churner reached is worth four wasted $15 offers. One caution: this example compares thresholds on the TEST set only to show the trade-off. To CHOOSE a threshold, use out-of-fold training probabilities, as the lesson project does.

GUIDED LAB

- Open `labs/module-03/l1_4-metrics/fraud_review_starter.py`. Run it once as it is: it prints the test-set size and six TODO reminders.

- The question: a reviewer checks each flagged card payment ($10), and every fraud that slips through costs about $375. Which threshold makes the review queue cheapest?
- TODO 1 and 2 (together): the confusion matrix in business words; ROC AUC next to average precision and the fraud rate. Write your prediction for TODO 2 first.
- TODO 3 to 6 (in pairs): the cost function, out-of-fold probabilities on the TRAINING payments, the cheapest threshold, one test-set comparison of four policies, and `FixedThresholdClassifier`.
- Done when your numbers match the "Expected when done" lines the starter prints.
- Finished early? The Stretch line, then `debug_metrics.py` (two bugs that stop it, two that do not). At home: `store_forecast_starter.py`.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code. A wrong prediction you can explain afterward is worth more than a right guess.

P1

Predict the output, then run it. (Six customers, two thresholds.)

```
import numpy as np
proba = np.array([0.9, 0.7, 0.4, 0.3, 0.2, 0.1])
truth = np.array([1, 0, 1, 1, 0, 0])
for t in [0.5, 0.25]:
    flagged = (proba >= t).astype(int)
    print(t, flagged.sum(), (flagged & truth).sum())
```

P2

Predict the output, then run it. (Predicting the mean every time.)

```
from sklearn.metrics import mean_absolute_error, r2_score
y = [2, 4, 6, 8]
print(r2_score(y, [5, 5, 5, 5]), mean_absolute_error(y, [5, 5, 5, 5]))
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Plan the Retention Campaign (lesson page, "Project: Plan the Retention Campaign"). In class: Steps 1 and 2. At home: Steps 3 to 6.

- ☐ Step 1: load and split, keeping `customer_id`. Check: `4500 1500 break-even probability 0.25`
- ☐ Step 2: out-of-fold forest probabilities. Check: `[0.185 0.005 0.191 0.008 0.494] mean 0.166`
- ☐ Step 3: the profit curve on the training customers: best threshold 0.26
- ☐ Step 4: the test set, once: four policies with precision, recall and profit
- ☐ Step 5: the 150-call list and its precision
- ☐ Step 6: `retention_calls.csv`: IDs and probabilities only

SELF-CHECK

Q1

A churn model's confusion matrix on a test set: TN 880, FP 60, FN 20, TP 40. What are its precision and recall?

A) precision 0.667, recall 0.400

B) precision 0.400, recall 0.667

C) precision 0.978, recall 0.936

D) precision 0.920, recall 0.400

Q2

What does this print?

```
from sklearn.metrics import precision_score, recall_score
y_true = [1, 0, 1, 1, 0, 0, 0, 1]
y_pred = [1, 1, 0, 1, 0, 0, 0, 0]
print(round(precision_score(y_true, y_pred), 3), recall_score(y_true, y_pred))
```

Q3

You want the decision threshold that maximizes the retention campaign's profit. Where should the probabilities you search over come from?

A) The test set: it is the most realistic data you have

B) The training rows, from the fitted model's own `predict_proba(X_train)`

C) The training rows, out of fold: `cross_val_predict(..., method="predict_proba")` or `TunedThresholdClassifierCV`

D) Nowhere: always use 0.5, because the model was trained for it

Q4

What does this print?

```
from sklearn.metrics import roc_auc_score
y = [0, 0, 1, 1]
print(roc_auc_score(y, [0, 1, 1, 1]), roc_auc_score(y, [0.2, 0.6, 0.7, 0.9]))
```

A) `0.75 1.0`

B) `1.0 1.0`

C) `0.5 1.0`

D) `ValueError`: ROC AUC needs probabilities

Q5

A retention offer costs $20 and reaching a real churner is worth $80. If the model's probabilities were perfectly calibrated, whom should Pinecrest contact?

A) Everyone predicted to churn at the default threshold of 0.5

B) Nobody: each offer costs money

C) The customers with a churn probability of at least 0.80

D) The customers with a churn probability of at least 0.25

Q6

What does this print?

```
from sklearn.metrics import mean_absolute_error, root_mean_squared_error
y_true = [10, 20, 30, 40]
y_pred = [12, 18, 30, 50]
print(mean_absolute_error(y_true, y_pred), round(root_mean_squared_error(y_tru
```

JOURNAL

Explain the smoke-alarm picture using the words precision, recall and threshold. Then write the four cells of the confusion matrix for a problem you care about (a medical test, a spam filter, a job screen) and put a rough cost on each. Where would you set the threshold, and who should get to decide?

2.1 Impute, Scale and Encode

BY THE END YOU CAN

- Fill missing values with SimpleImputer, and keep the fact that a value was missing with `add_indicator=True`
- Choose between StandardScaler, MinMaxScaler, RobustScaler and a log transform, and explain what each one changes
- Encode categories with OneHotEncoder (including categories never seen in training) and ordered categories with OrdinalEncoder
- Build a ColumnTransformer that sends each group of columns to the right transformer, fitted on the training data only
- Train a model on all 16 usable churn columns, not just the 10 clean numeric ones

KEY IDEAS

- **Mise en place.** Each kind of column gets its own treatment: gaps → imputer, units → scaler, words → encoder. The recipe is learned from the training rows, then applied unchanged to the test rows and every new customer.
- **Ask what a gap means before filling it.** Customers with no email count churned at 25.3% against 15.6%: the gap is a clue.
- **SimpleImputer** fills with a learned value (`statistics_`): median for skewed numbers, `most_frequent` or a constant such as "unknown" for text. `add_indicator=True` keeps a 0/1 "was missing" column.
- **Scalers change units, not shape.** StandardScaler (mean 0, sd 1), MinMaxScaler (0 to 1), RobustScaler (median 0, interquartile range 1; resists outliers). A log (`np.log1p`) changes the shape. Trees need neither.
- **OneHotEncoder(sparse_output=False, handle_unknown="ignore")**: one 0/1 column per category; an unseen category becomes all zeros; a gap becomes its own column. Never `pd.get_dummies` on new data.
- **OrdinalEncoder** only for ordered categories, always with the order spelled out: `categories=[["Basic", "Silver", "Gold"]]`. The default is alphabetical.
- **ColumnTransformer**: a list of (name, transformer, columns); branches run side by side, unlisted columns are dropped. Steps that must run one after another go in a `make_pipeline` inside one branch.
- **fit_transform on train, transform on everything else.** Check it: scaled training columns have a mean of exactly 0; test columns only close to 0.

◆ WORKED EXAMPLE: THE PRODUCT CATALOG, READY FOR A MODEL

A different Pinecrest table, the product catalog, prepared with the same tools. No model today: the goal is a numeric table and a check that a brand-new product still gets a row.

```
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import MinMaxScaler, OneHotEncoder
products = pd.read_csv("data/products_ml.csv")
print(products.shape)
print(products[["brand", "price", "launch_year", "primary_activity"]].head(3))
```

```
(200, 8)
            brand   price  launch_year primary_activity
0  Larkspur Ridge  220.95         2020           hiking
1  Larkspur Ridge  164.95         2025          running
2  Larkspur Ridge  165.95         2023         climbing
```

Two numbers on very different scales, and two text columns with no order. MinMaxScaler puts both numbers on 0 to 1; OneHotEncoder makes one column per brand and per activity.

```
prep = ColumnTransformer([
    ("nums", MinMaxScaler(), ["price", "launch_year"]),
    ("cats", OneHotEncoder(sparse_output=False, handle_unknown="ignore"), ["brand"
]).set_output(transform="pandas")
table = prep.fit_transform(products)
print(table.shape)
print(table.columns[:5].tolist())
```

```
(200, 16)
['nums__price', 'nums__launch_year', 'cats__brand_Cedar Notch', 'cats__brand_Gran
```

Eight columns became 16: two scaled numbers, 8 brands and 6 activities. The name and the category were not listed, so they were dropped. Now a product launched next year by a brand the catalog has never seen:

```
new = pd.DataFrame({"price": [99.95], "launch_year": [2026], "brand": ["Brand-New
                    "primary_activity": ["hiking"]})
row = prep.transform(new)
print(row[["nums__price", "nums__launch_year"]].round(3).to_dict("records"))
print("brand columns set:", int(row.filter(like="cats__brand").sum(axis=1).iloc[0]
      "| activity columns set:", int(row.filter(like="cats__primary_activity").sum
```

```
[{'nums__price': 0.127, 'nums__launch_year': 1.167}]
brand columns set: 0 | activity columns set: 1
```

Two things to notice. The launch year scales to 1.167, above 1: MinMaxScaler learned 2019 to 2025 from the training table and does not clip new values. And the unknown brand sets no brand column at all (all zeros, thanks to `handle_unknown="ignore"`), while the known activity, hiking, sets exactly one.

GUIDED LAB

- Open `labs/module-03/l2_1-preprocessing/fraud_columns_starter.py`. Run it once as it is.
- The question: does the product category help the fraud model, and does the model survive next month's feed (a category never sold before, a payment with no account age)?
- TODO 1 (together): the fraud rate by category on the training rows.
- TODO 2 to 6 (in pairs): the ColumnTransformer, the model with and without the category (judged with ROC AUC and average precision), what the preprocessor learned, and three new payments.
- Done when your numbers match the "Expected when done" lines.
- Finished early? `debug_preprocess.py`. At home: `demand_categories_starter.py`.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code. A wrong prediction you can explain afterward is worth more than a right guess.

P3

Predict the output, then run it. (Two of the three known values are 25-34.)

```
import numpy as np
from sklearn.impute import SimpleImputer
imp = SimpleImputer(strategy="most_frequent").fit([["25-34"], [np.nan], ["25-3
print(imp.statistics_, imp.transform([[np.nan]]).ravel())
```

P4

Predict the output, then run it. (Median 3; the middle half runs from 2 to 4.)

```
from sklearn.preprocessing import RobustScaler
print(RobustScaler().fit_transform([[1], [2], [3], [4], [100]]).ravel())
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

A Model-Ready Customer Table (lesson page, "Project: A Model-Ready Customer Table"). In class: Steps 1 to 3. At home: Steps 4 to 6.

- ☐ Step 1: split the whole table, keep the IDs. Check: `16 features; 4500 train rows; 1500 test rows`
- ☐ Step 2: the gap audit on the training rows (which column deserves an indicator?)
- ☐ Step 3: the preprocessor. Check: `(4500, 40) (1500, 40)`
- ☐ Step 4: three checks: no gaps, train columns centered exactly, test columns only close
- ☐ Step 5: baseline against logistic regression, accuracy and ROC AUC
- ☐ Step 6: three new customers scored; `new_customer_scores.csv`

SELF-CHECK

Q7

What does this print?

```
import pandas as pd
from sklearn.preprocessing import OneHotEncoder
enc = OneHotEncoder(sparse_output=False, handle_unknown="ignore")
enc.fit(pd.DataFrame({"store": ["Bend", "Boise", "Bend"]}))
print(enc.transform(pd.DataFrame({"store": ["Boise", "Reno"]})))
```

A) `[[0. 1. 0.] [0. 0. 1.]]` (a new Reno column)

B) `ValueError`: Found unknown categories ['Reno']

C) `[[0. 1.] [0. 0.]]`

D) `[[1. 0.] [0. 0.]]`

Q8

What does this print?

```
import numpy as np
from sklearn.impute import SimpleImputer
imp = SimpleImputer(strategy="mean", add_indicator=True).fit([[2.0], [np.nan],
print(imp.transform([[np.nan], [1.0]]))
```

Q9

After `StandardScaler`, `days_since_last_order` still has a skew of 1.71, the same as the raw column. Why?

A) Scaling changes the units, not the shape; a log transform such as `np.log1p` changes the shape

B) The scaler was fitted on the test rows instead of the training rows

C) StandardScaler only removes skew when there are no outliers

D) Skew cannot be measured on scaled data

Q10

What does this print?

```
from sklearn.preprocessing import OrdinalEncoder
enc = OrdinalEncoder().fit([["Small"], ["Medium"], ["Large"]])
print(enc.transform([["Small"], ["Large"]]).ravel())
```

A) `[0. 2.]`

B) `[2. 1.]`

C) `[1. 0.]`

D) `[2. 0.]`

Q11

In a `ColumnTransformer`, the `email_opens_90d` column needs its gaps filled AND then scaling. How do you do it?

A) Two branches on the same column: one with the imputer, one with the scaler

B) One branch whose transformer is `make_pipeline(SimpleImputer(...), StandardScaler())`

C) Put the scaler in the branch and fill the gaps in pandas on the whole table before splitting

D) Set `remainder="passthrough"` so the scaler runs after the imputer

Q12

What does this print?

```
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
df = pd.DataFrame({"spend": [10.0, 20.0, 30.0], "store": ["Bend", "Boise", "Be
                   "customer_id": ["C1", "C2", "C3"]})
ct = ColumnTransformer([("num", StandardScaler(), ["spend"]),
                        ("cat", OneHotEncoder(sparse_output=False), ["store"])
print(ct.fit_transform(df).shape, ct.get_feature_names_out().tolist())
```

◆ JOURNAL

Pick three columns from the churn table: one with gaps, one in dollars and one with text. For each, write the transformer you chose, what it learns during fit, and what would go wrong if it learned that from the test set instead. Then write one sentence about the missing email counts: what did the gap itself tell you?

EXIT TICKET

Three questions, no notes. Write your answers, hand them in or keep them: they tell you and your instructor what to revisit.

E1

A model's confusion matrix: TN 900, FP 100, FN 30, TP 70. Compute its precision and recall, and say each in one business sentence.

E2

Why choose a decision threshold from out-of-fold training probabilities instead of the test set?

E3

Next month's customer comes from the new Reno store and did not give an age. What does each part of your fitted preprocessor do with them?

NOTES

HOMEWORK CHECKLIST

- ☐ Finish the retention-campaign plan (Steps 3 to 6) and the model-ready customer table (Steps 4 to 6)
- ☐ Lesson 1.4 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Lesson 2.1 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Both Quick Quizzes on the lesson pages
- ☐ Both journal entries in `journal.md`
- ☐ "Before the Next Lesson" (1.4 and 2.1): the unused churn columns, one dashboard metric and why, print `preprocess`, and which objects must be refitted inside every fold
- ☐ Optional: `store_forecast_starter.py`, `demand_categories_starter.py`, `debug_metrics.py`, `debug_preprocess.py`
- ☐ Bring to Session 4: a READY laptop, `pinecrest-ml` with `data/` and `labs/`, my 2.1 project notebook,

this workbook and my journal

GLOSSARY

TERM	MEANING
confusion matrix	The four counts TN, FP, FN, TP: rows are the truth, columns the prediction
positive class	The class you are looking for (churned, fraud), coded 1 or named with `pos_label`
precision	TP / (TP + FP): of those flagged, the share that really are positive
recall	TP / (TP + FN): of the real positives, the share flagged (also sensitivity, true positive rate)
F1 / F-beta	Harmonic mean of precision and recall; beta above 1 weighs recall more
decision threshold	The probability above which a case is flagged; 0.5 by default, a business decision
ROC AUC	Chance a random positive outranks a random negative; 0.5 random, 1.0 perfect
average precision	Summary of the precision-recall curve; its random baseline is the positive rate
break-even probability	cost / value: the probability at which contacting someone pays for itself
out-of-fold prediction	A prediction for a training row from a fold model that did not see that row (`cross_val_predict`)
MAE / RMSE / R^2	Average miss in units / root of the mean squared miss / share of variation explained compared with the mean
imputation	Filling a gap with a value learned from the training data (SimpleImputer)
missing indicator	A 0/1 column that remembers which values were filled (`add_indicator=True`)
one-hot encoding	One 0/1 column per category (OneHotEncoder)
ordinal encoding	One column of ordered codes 0, 1, 2 (OrdinalEncoder, order spelled out)
ColumnTransformer	Sends each list of columns to its own transformer and puts the results side by side

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 4: Pipelines and Creating Features

Your workbook for Session 4: a warm-up from Session 3, the key ideas of Lessons 2.2 and 2.3, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 2.2–2.3 | Session 4 · 3 hours | Name: _____

SESSION GOALS

- Build any model that needs preprocessing as ONE pipeline, read and change any setting inside it, and save it to a file.
- Explain, with a number, why every step that learns from data must be fitted inside the cross-validation folds.
- Turn what you see in the training rows (a jump, a U-shape, an effect that differs by group) into features a linear model can use.
- Build date, cycle, lag and rolling features that never look into the future, and measure every idea before keeping it.

WARM-UP: FROM SESSION 3

Five questions on Lessons 1.4 and 2.1, no notes. Your instructor reveals the answers.

W1

Pinecrest's care team calls 150 customers and 103 of them really were about to churn. Which metric is 103 / 150, and what does it answer? (Lesson 1.4)

W2

What does this print? (Lesson 1.4)

```
from sklearn.metrics import precision_score, recall_score
y_true = [1, 1, 1, 0, 0, 0, 0, 0]
y_pred = [1, 1, 0, 1, 0, 0, 0, 0]
print(round(precision_score(y_true, y_pred), 3), round(recall_score(y_true, y_pred)
```

W3

Lesson 1.4's project chose a threshold of 0.26. Why from out-of-fold probabilities on the training rows, and never from the test set? (Lesson 1.4)

W4

What does this print? (Lesson 2.1)

```
from sklearn.preprocessing import OneHotEncoder
enc = OneHotEncoder(sparse_output=False, handle_unknown="ignore").fit([["Bend"], ["
print(enc.transform([["Reno"], ["Boise"]]))
```

W5

Lesson 2.1 asked: to cross-validate its churn model, which objects must be refitted inside every fold, and in which order? (Lesson 2.1)

LESSONS

2.2 Pipelines

BY THE END YOU CAN

- Build a Pipeline with `make_pipeline` or with named steps, and explain what `fit` and `predict` do to each step
- Reach inside a fitted pipeline (`named_steps`, indexing, slicing) to read learned values
- Cross-validate a whole pipeline, and show with a target-based encoding why preprocessing outside the folds leaks
- Change any setting with the `step__parameter` naming, swap a whole step, and run a first small grid search
- Add a custom step with `FunctionTransformer`, and save and reload a fitted pipeline with joblib

KEY IDEAS

- **The coffee machine.** A pipeline is transformers in order plus one model at the end, with the same `fit`, `predict`, `predict_proba` and `score` as a single model.
- **fit vs predict.** `fit` calls `fit_transform` on every step but the last, then `fit`; `predict` calls only `transform`, then `predict`. Nothing is refitted on the rows being predicted.
- **Looking inside.** `pipe.named_steps["prep"]`, `pipe[-1]` (the model), `pipe[:-1]` (everything before it). `named_transformers_` holds a ColumnTransformer's fitted branches.
- **Leaks.** Any step that learns from data must be fitted inside each fold. Steps that learn from y (target encoding, selection, resampling) leak the most: 0.930 leaky vs 0.586 honest for the signup-date rate.
- **Names.** `step__parameter`, deeper `step__branch__substep__parameter`, for `set_params` and grid searches. `make_pipeline` names steps after their classes; `Pipeline([("prep", ...), ("model", ...)])` uses yours.
- **Swap and search.** `clone(pipe).set_params(model=...)` swaps a whole step; `GridSearchCV(pipe, param_grid=..., cv=cv)` cross-validates every combination without leaks.
- **Custom and saved.** `FunctionTransformer(func)` makes a stateless step; `joblib.dump` / `joblib.load` save the whole fitted pipeline (load only files you trust).
- **What a pipeline cannot do.** It separates rows, not columns: a column from the future in your feature list still leaks.

WORKED EXAMPLE: A PIPELINE THAT FILLS A GAP

A three-column churn model with a gap to fill, as one pipeline with named steps. First, cross-validate the whole thing: the imputer and the scaler are refitted in every fold.

```
import numpy as np
import pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score, train_test_s
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
customers = pd.read_csv("data/customers_churn.csv")
cols = ["days_since_last_order", "discount_share", "email_opens_90d"]
X_train, X_test, y_train, y_test = train_test_split(
    customers[cols], customers["churned"], test_size=0.25, stratify=customers["chu
pipe = Pipeline([
    ("fill", SimpleImputer(strategy="median")),
    ("scale", StandardScaler()),
    ("model", LogisticRegression()),
])
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(pipe, X_train, y_train, cv=cv, scoring="roc_auc")
print(f"CV ROC AUC {scores.mean():.3f} +/- {scores.std():.3f}")
```

```
CV ROC AUC 0.813 +/- 0.024
```

Now fit it once on all training rows and score two customers who are in no file, both with an unknown email count.

```
pipe.fit(X_train, y_train)
print("medians learned:", pipe.named_steps["fill"].statistics_)
new = pd.DataFrame({"days_since_last_order": [20, 220], "discount_share": [0.10, 0
                    "email_opens_90d": [np.nan, np.nan]})
print(pipe[:-1].transform(new).round(2))
print("churn probability:", pipe.predict_proba(new)[:, 1].round(3))
```

```
medians learned: [52.     0.141  6.   ]
[[-0.73 -0.61 -0.22]
 [ 1.57  1.63 -0.22]]
churn probability: [0.043 0.669]
```

Read it from the inside out. The imputer learned three medians from the training rows (52 days, a discount share of 0.141, 6 opens). Both new customers have no email count, so both get 6, which scales to -0.22: the same value for both, because nothing is refitted on new rows. `pipe[:-1]` shows the data exactly as the

model sees it. The first customer (20 days, light discounts) gets 0.043; the second (220 days, half of spend on discount) 0.669.

GUIDED LAB

- **Lab A:** `labs/module-04/l2_2-pipelines/fraud_pipeline_starter.py`. A pipeline for card fraud: build `prep` + `model`, compare it with a baseline pipeline by cross-validation (ROC AUC and average precision), reach inside, turn `model__C`, test once, save, reload and score three raw payments. Before TODO 2, write down what you think the baseline's average precision will be.
- **Lab B (pairs):** `labs/module-04/l2_2-pipelines/leak_size_starter.py`. Three steps done before cross-validation and inside a pipeline. The one not typing predicts each gap out loud.
- Done when your numbers match the "Expected when done" lines.
- Finished early? `labs/module-04/l2_2-pipelines/debug_pipeline.py`: three bugs that stop it, one that does not.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code. A wrong prediction you can explain afterward is worth more than a right guess.

P1

Predict the output, then run it. (make_pipeline names steps after their classes.)

```
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
pipe = make_pipeline(SimpleImputer(), StandardScaler(), LogisticRegression())
print(list(pipe.named_steps))
print(len(pipe[:-1]))
```

P2

Predict the output, then run it. (The imputer was fitted on three training values.)

```
from sklearn.impute import SimpleImputer
imp = SimpleImputer(strategy="median").fit([[1.0], [2.0], [9.0]])
print(imp.transform([[None], [100.0]]).ravel())
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

A deployable churn pipeline (lesson page, "Project: A Deployable Churn Pipeline"). In class: Steps 1 to 3. At home: Steps 4 to 6.

- ☐ Step 1: load, split, columns, `cv`. Check: `(4500, 16) (1500, 16)`
- ☐ Step 2: one preprocessor, `churn_pipeline(model)`, three candidates
- ☐ Step 3: CV baseline 0.500, logistic regression 0.833 ± 0.021, gradient boosting 0.861 ± 0.013; chosen: gradient boosting
- ☐ Step 4: test ROC AUC 0.863; 42 real churners in the top 50 (random list 8.3)
- ☐ Step 5: `churn_pipeline.joblib` reloads; 16 columns expected
- ☐ Step 6: two raw records scored (N1 0.866, N2 0.002); `monthly_churn_scores.csv`

SELF-CHECK

Q1

A fitted pipeline `make_pipeline(StandardScaler(), LogisticRegression())` is asked to `predict_proba(X_new)`. What does the scaler step do with `X_new`?

A) `fit_transform`: the new rows are centered on their own mean

B) `transform` only, with the means and standard deviations it learned during `fit`

C) Nothing: scaling only happens during fit

D) It refits on the training rows and the new rows together

Q2

Which name sets the imputer's strategy in this pipeline?

```
pipe = Pipeline([
    ("prep", ColumnTransformer([
        ("num", make_pipeline(SimpleImputer(), StandardScaler()), ["email_oper
    ])),
    ("model", LogisticRegression()),
])
```

A) `prep__simpleimputer__strategy`

B) `num__simpleimputer__strategy`

C) `prep_num_simpleimputer_strategy`

D) `prep__num__simpleimputer__strategy`

Q3

What happens when this runs?

```
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
pipe = make_pipeline(StandardScaler(), LogisticRegression())
pipe.set_params(model__C=0.1)
```

A) `ValueError`: `Invalid parameter` `'model'`: make_pipeline named the step `logisticregression`

B) It sets C on the logistic regression

C) `TypeError`: C can only be set when the model is created

D) Nothing: the unknown setting is silently ignored

Q4

Which of these steps could be applied to all training rows BEFORE cross-validation without changing the cross-validated score at all?

A) `SimpleImputer(strategy="median")`

B) `TargetEncoder()` on a high-cardinality column

C) `FunctionTransformer(np.log1p)`

D) `SelectKBest(k=10)` (keeps the columns most related to y)

Q5

What does this print?

```
import numpy as np
from sklearn.preprocessing import FunctionTransformer
log = FunctionTransformer(np.log1p).fit([[0.0], [9.0]])
print(log.transform([[99.0]]).round(3))
```

Q6

The IT team reloads `churn_pipeline.joblib` and sends next month's records, but their export leaves out the `email_opens_90d` column. What happens?

A) The imputer fills the whole column with the training median

B) The pipeline uses the other 15 columns and ignores the missing one

C) It predicts the baseline churn rate for every customer

D) `ValueError: columns are missing: {'email_opens_90d'}`: send the column, empty (NaN) where unknown

◆ JOURNAL

Explain the signup-date leak to a colleague who has never heard of cross-validation: why did the score look like 0.93, and why was it really about 0.57? Then list two kinds of leak a pipeline prevents and one kind it

cannot prevent, with a Pinecrest example of each.

2.3 Creating Features

BY THE END YOU CAN

- Explain why a linear model cannot use a threshold, a U-shape or an interaction unless you build it as a feature
- Create flags, ratios and interaction terms with pandas, choosing them from the training data only
- Extract calendar parts and cyclical (sine and cosine) features from a date column
- Build lag and rolling-window features per group with `shift`, so no row sees its own future
- Put feature creation inside a Pipeline with `FunctionTransformer` and measure the lift with cross-validation

KEY IDEAS

- **The interviewer and the calculator.** A linear model only adds weighted columns. A threshold, a U-shape or an interaction exists for it only if you build it as a column.
- **Look before you build, on the training rows only.** Groupbys by ranges (`pd.cut`) and pivot tables reveal jumps (recency near 150 days), U-shapes (tenure) and interactions (tickets without the app).
- **Flags, products and ratios.** A flag `(x > t).astype(int)`, an interaction `a * (1 - b)`, a ratio over a column that is never 0. Write them in one pure function (`df.copy()`).
- **Measure inside a pipeline.** `FunctionTransformer(add_features)` before the scaler; cross-validate against the plain model. Five churn features: 0.829 → 0.883.
- **Dates.** `parse_dates`, then `.dt.month`, `.dt.dayofweek`, date differences. Cycles: `sin` and `cos` of `2π · month / 12`, both columns.
- **Combined categories.** `category + "-" + month` gives each category its own season (demand MAE 11.23 → 9.39).
- **History.** Per series and past only: `groupby(...).shift(1)`, `shift(1).rolling(k).mean()`. Ask: on the day of the prediction, would this number exist?
- **Aggregates.** Compute group averages of the target from the training rows only (or per fold, with TargetEncoder). A log target turns multiplicative effects into additive ones.

WORKED EXAMPLE: A DATE FEATURE THAT WAS ALREADY THERE

Lesson 2.3 says a column like `tenure_months` is usually born from two dates. Check it, then try a calendar feature from `signup_date` on the training rows.

```
import pandas as pd
from sklearn.model_selection import train_test_split
customers = pd.read_csv("data/customers_churn.csv", parse_dates=["signup_date"])
train, test = train_test_split(customers, test_size=0.25, stratify=customers["chur
train = train.copy()
snapshot = pd.Timestamp("2025-06-30")
train["days_since_signup"] = (snapshot - train["signup_date"]).dt.days
print(train[["signup_date", "days_since_signup", "tenure_months"]].head(3))
print("correlation with tenure:", round(train["days_since_signup"].corr(train["ten
```

```
     signup_date  days_since_signup  tenure_months
1556  2025-05-11                 50              1
965   2021-10-28               1341             44
5145  2025-02-25                125              4
correlation with tenure: 1.0
```

A correlation of 1.0: the days since signup ARE tenure, in other units. A feature you build may already be in the table. Next, the signup month:

```
train["signup_month"] = train["signup_date"].dt.month
by_month = train.groupby("signup_month").agg(
    churn=("churned", "mean"), new_share=("tenure_months", lambda t: (t < 6).mean(
print(by_month.round(2).T)
```

```
signup_month    1     2     3     4     5     6     7     8     9     10    11    
churn         0.19  0.20  0.20  0.19  0.24  0.11  0.09  0.12  0.14  0.14  0.11  0
new_share     0.40  0.44  0.42  0.40  0.38  0.00  0.00  0.00  0.00  0.00  0.00  0
```

Signup month looks like a strong signal: about 20% churn for January to May, 9 to 15% for the other months. But look at `new_share`: every customer with under six months of tenure signed up between January and May 2025. Remove them:

```
settled = train[train["tenure_months"] >= 6]
rates = settled.groupby("signup_month")["churned"].mean()
print(f"customers with 6+ months: churn by signup month from {rates.min():.3f} to
```

```
customers with 6+ months: churn by signup month from 0.091 to 0.153
```

Among customers with six months or more, the spread shrinks to 9 to 15%, close to what chance produces in groups of about 300 to 450 (the 2,000 customers who joined in a given month across eight years). The month was mostly the new-customer effect in disguise. Before you keep a feature, ask what it might be standing in for, and measure it next to the features you already have.

GUIDED LAB

- **Lab A:** `labs/module-04/l2_3-creating-features/night_features_starter.py`. In Session 1 the fraud model gave `hour` a backwards weight. Look at the training rows, build night features, put them in the pipeline, and measure.
- **Lab B (pairs):** `labs/module-04/l2_3-creating-features/daily_sales_features_starter.py`. Tomorrow's store sales: day of the week, lags per store, three linear models. Swap the keyboard at TODO 4.
- Done when your numbers match the "Expected when done" lines.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code. A wrong prediction you can explain afterward is worth more than a right guess.

P3

Predict the output, then run it. (An interaction term.)

```
import pandas as pd
df = pd.DataFrame({"tickets": [0, 2, 3, 1], "app_user": [1, 1, 0, 0]})
df["tickets_no_app"] = df["tickets"] * (1 - df["app_user"])
print(df["tickets_no_app"].tolist())
```

P4

Predict the output, then run it. (June 28, 2025 is a Saturday; .dt.dayofweek counts Monday as 0.)

```
import pandas as pd
days = pd.to_datetime(pd.Series(["2025-06-28", "2025-06-30", "2025-07-04"]))
print(days.dt.dayofweek.tolist(), days.dt.month.tolist())
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The weekly demand forecaster (lesson page, "Project: The Weekly Demand Forecaster"). In class: Steps 1 to 3. At home: Steps 4 to 6.

- ☐ Step 1: features on the full table, split by year. Check: `5088 4896 0 missing lags in test`, baseline MAE 17.13
- ☐ Step 2: raw columns 11.28; + month 11.23 (why so little?)
- ☐ Step 3: + category × month 9.39
- ☐ Step 4: log target 7.92
- ☐ Step 5: + lags 7.92 (a non-result worth recording)
- ☐ Step 6: boosting on raw columns 8.61; the scoreboard chart; `demand_scoreboard.csv`

SELF-CHECK

Q7

What does this print?

```
import pandas as pd
df = pd.DataFrame({"store": ["A", "A", "B", "B", "A"], "units": [5, 7, 9, 4, 6
print(df.groupby("store")["units"].shift(1).tolist())
```

Q8

Churn is 32% for customers under 6 months, about 8 to 11% in the middle years and 21.5% above 72 months, yet a logistic regression on the raw columns gives tenure only a small weight. Why?

A) Tenure does not really matter for churn

B) Tenure was not scaled

C) A straight line through a U-shape is nearly flat; flags such as new_customer and veteran let the model express it

D) Tenure is redundant with `signup_date`

Q9

What does `tickets_no_app = support_tickets_12m * (1 - app_user)` let a linear model do?

A) Give support tickets their own weight for customers without the app

B) Remove app users from the training data

C) Count every app user's tickets twice

D) Replace `app_user`, which can then be dropped

Q10

What does this print?

```
import numpy as np
months = np.array([3, 6, 12])
print(np.cos(2 * np.pi * months / 12).round(2))
```

Q11

An average of units sold per product, store and month computed from BOTH years forecasts 2025 with MAE 9.87; computed from 2024 only, 11.36. Why is 9.87 not a real improvement?

A) Averages over more rows always overfit

B) The 2025 weeks being predicted helped compute their own feature

C) MAE rewards forecasts that are too low

D) 2025 has fewer weeks than 2024

Q12

What does this print?

```
import pandas as pd
df = pd.DataFrame({"returns": [1, 0, 2], "tickets": [2, 0, 0]})
print((df["returns"] / df["tickets"]).tolist())
```

A) `[0.5, 0.0, 0.0]`

B) A `ZeroDivisionError`

C) `[0.5, nan, inf]`

D) `[0.5, nan, nan]`

◆ JOURNAL

Explain the interviewer and the calculator in your own words, using one churn feature and one demand feature from today as examples. Then copy your project scoreboard and mark the two lines where an idea about the business, not a better algorithm, made the biggest difference. What was the idea in each case?

EXIT TICKET

Three questions, no notes.

E1

Name two things a Pipeline does that you had to remember by hand in Lesson 2.1, and one kind of leak it cannot prevent.

E2

The signup-date churn rate scored 0.930 in cross-validation and 0.571 on the test set. Why, and what fixed the measurement?

E3

Give one feature you built today, the shape or interaction it lets a linear model express, and how you knew it helped.

NOTES

HOMEWORK CHECKLIST

- ☐ Finish the deployable churn pipeline (Steps 4 to 6) and the weekly demand forecaster (Steps 4 to 6)
- ☐ Lesson 2.2 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Lesson 2.3 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Both Quick Quizzes on the lesson pages
- ☐ Both journal entries
- ☐ Keep `churn_pipeline.joblib`, `demand_scoreboard.csv` and both project notebooks
- ☐ Rewrite one Module 1 model as a pipeline: does its score change?
- ☐ Write down three churn columns I suspect carry the same information as another column (for Lesson 2.4)
- ☐ Ten minutes with Lesson 1.4's Key Takeaways (precision, recall, average precision) before Session 5

- ☐ Bring to Session 5: my environment, `pinecrest-ml` with `data/` and `labs/`, my 2.3 notebook (`add_churn_features`), my redundancy list, this workbook and my journal

GLOSSARY

TERM	MEANING
pipeline	Transformers in order plus one final model, used as a single estimator (`Pipeline`, `make_pipeline`)
step name	The label of a step: yours in `Pipeline([...])`, the lowercase class name in `make_pipeline`
`step__parameter`	The address of a setting inside a pipeline, levels joined by two underscores
`named_transformers_`	A fitted ColumnTransformer's branches, by name
clone	An unfitted copy of an estimator with the same settings
grid search	`GridSearchCV`: cross-validate every combination of settings and refit the best
target encoding	Replacing a category with the average target for that category; fitted inside the folds (`TargetEncoder`)
stateless / stateful step	A step that learns nothing (a fixed function) / one that learns from the data and must be fitted in each fold
engineered feature	A new column built from existing ones to give the model a question it could not ask
flag	A 0/1 column for a condition, such as `lapsed_150`
interaction term	A product of columns whose effect depends on another, such as `tickets_no_app`
cyclical encoding	`sin` and `cos` of a periodic value so December sits next to January
lag / rolling feature	A past value of the target / a summary of a window of past values, per series, via `shift`
aggregate feature	A group summary (such as an average) looked up for each row; from training rows only

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 5: Feature Selection and Imbalanced Classes

Your workbook for Session 5: a warm-up on pipelines and features, the key ideas of Lessons 2.4 and 2.5, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 2.4–2.5 | Session 5 · 3 hours | Name: _____

SESSION GOALS

- Find redundant columns by asking how each one was made, and confirm with correlations and exact checks.
- Use a filter, a wrapper and an L1 penalty, and say what each one can miss.
- Keep feature selection inside the pipeline, and choose the smallest set within a tolerance of the best score.
- Judge a rare-class model by its ranking (average precision against the base rate), not by accuracy.
- Explain what class weights and resampling really change, and set a threshold from the cost of each kind of mistake.

WARM-UP: PIPELINES AND FEATURES

Five questions on Lessons 2.2 and 2.3. Answer alone, then compare with a neighbor.

W1

A pipeline is `make_pipeline(SimpleImputer(), StandardScaler(), LogisticRegression())`. What does `pipe.predict(X_new)` do to each of the three steps?

W2

In Lesson 2.2 the signup-date churn rate scored CV ROC AUC 0.930 when computed with pandas, and 0.586 as a `TargetEncoder` inside the pipeline. Which is honest, and why was the other so high?

W3

What does this print?

```
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
pipe = make_pipeline(StandardScaler(), LogisticRegression())
print([name for name in pipe.get_params() if name.endswith("__C")])
```

W4

`add_churn_features` builds `tickets_no_app = support_tickets_12m * (1 - app_user)`. What is it for an app user with 3 tickets, and why build it?

W5

Why can `FunctionTransformer(add_churn_features)` be fitted outside the folds without leaking, while `TargetEncoder` cannot?

LESSONS

2.4 Feature Selection

BY THE END YOU CAN

- Explain what feature selection buys you (simpler, cheaper, sturdier models) and what it usually does not (big accuracy gains)
- Find redundant features with correlations, exact relationships and cross-tabulations
- Use filter (`SelectKBest`), wrapper (`RFECV`, `SequentialFeatureSelector`) and embedded (L1, `SelectFromModel`) methods, and say what each can miss
- Keep selection inside a `Pipeline` so cross-validation stays honest, and show what happens when it is not
- Choose the smallest feature set whose cross-validated score is within a tolerance of the best

KEY IDEAS

- **The ultralight backpack.** Redundant items (two stoves) and irrelevant ones (the unused chair) go home; the goal is the lightest pack that still finishes the trail: the smallest feature set that keeps the score.
- **Selection buys simplicity, not accuracy.** Fewer inputs to collect, explain and monitor; on a good set of columns the score barely moves.
- **Ask how a column was made.** Averages, ratios, tiers and lookups computed from other columns are candidates to drop. Correlation misses relationships between three columns (spend = orders × average order value).
- **Filters** (`SelectKBest` with `f_classif` or `mutual_info_classif`) score each column alone: fast, but blind to the model and to teamwork. F sees differences in means; mutual information sees any dependence but is noisy.
- **Wrappers** (`RFECV`, `SequentialFeatureSelector`) try subsets with the real model: they find columns that matter together, at the cost of many fits.
- **Embedded:** an L1 penalty (`l1_ratio=1`, `solver="liblinear"`) zeros weak weights; smaller `C` means more zeros. Always scale first.
- **Selection that sees the labels is learning.** Keep it inside the pipeline, or cross-validation lies: about 80% on pure noise.
- **The tolerance rule:** read the curve, not the winner; take the smallest set within a small tolerance of the best, and test once. "Dropped" means "adds nothing given the others", not "irrelevant".

WORKED EXAMPLE: RELATED OR REDUNDANT?

Before any algorithm, ask how each column was made. Pinecrest's weekly demand table (Lesson 3.1 and later) hides three columns that the others already determine.

```
import pandas as pd
demand = pd.read_csv("data/weekly_demand.csv")
print("floor areas per store  :", demand.groupby("store")["store_sqft"].nunique().
print("list prices per product:", demand.groupby("product_id")["list_price"].nunic
rebuilt = (demand["list_price"] * (1 - demand["discount_pct"] / 100)).round(2)
print("largest gap between price and the rebuilt price:", (demand["price"] - rebui
```

```
floor areas per store  : {'Asheville': 1, 'Bend': 1, 'Boise': 1, 'Flagstaff': 1}
list prices per product: 1
largest gap between price and the rebuilt price: 0.0
```

Each store has exactly one floor area, and each product one list price, so once a model knows the store and the product, `store_sqft` and `list_price` add nothing. And `price` is `list_price × (1 - discount_pct / 100)` to the cent. Related is not the same as redundant, though:

```
print(pd.crosstab(demand["discount_pct"], demand["promo"]))
```

```
promo            0    1
discount_pct
0             4950  266
10             789   38
15             805   40
20             248  692
25             274  673
30             273  814
40              31   91
```

Promotions happen mostly at discounts of 20% or more, but there are 382 promo weeks below 20% off and hundreds of deep-discount weeks without a promo. `promo` carries information that `discount_pct` does not: keep both and let cross-validation judge.

GUIDED LAB

- **Lab A:** `labs/module-05/l2_4-feature-selection/lean_fraud_starter.py`. Fourteen fraud columns (nine raw, five engineered). TODO 1 to 4 in class: correlations, the F-test, CV average precision, forward selection. TODO 5 (tolerance rule and one test) if time allows.
- **Lab B (pairs):** `labs/module-05/l2_4-feature-selection/vendor_columns_starter.py`. A vendor's 1,000 columns on a 300-customer pilot. One types, one predicts each number out loud.
- Done when your numbers match the "Expected when done" lines the starters print.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code. A wrong prediction you can explain afterward is worth more than a right guess.

P1

Predict the output, then run it. (Which two columns differ most between the classes?)

```
import pandas as pd
from sklearn.feature_selection import SelectKBest, f_classif
X = pd.DataFrame({"a": [1, 2, 3, 4, 5, 6], "b": [5, 1, 4, 2, 6, 3], "c": [0, 0
y = [0, 0, 0, 1, 1, 1]
print(SelectKBest(f_classif, k=2).fit(X, y).get_feature_names_out())
```

P2

Predict the output, then run it. (y is exactly twice the first column.)

```
import numpy as np
from sklearn.linear_model import Lasso
X = np.array([[1, 0], [2, 1], [3, 0], [4, 1]])
y = np.array([2, 4, 6, 8])
print(Lasso(alpha=0.1).fit(X, y).coef_.round(2))
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The lean churn model (lesson page, "Project: The Lean Churn Model"). In class: Steps 1 to 3. At home: Steps 4 to 6.

- ☐ Step 1: 16 candidates; gaps in `email_opens_90d` (419) and `age_group` (359)
- ☐ Step 2: `build_model` for any list of columns. Check: all 16 columns, CV ROC AUC 0.8324
- ☐ Step 3: without the four derived columns: 0.8305
- ☐ Step 4: forward selection by hand, ten steps, starting with `days_since_last_order`

- ☐ Step 5: the tolerance rule gives 6 columns
- ☐ Step 6: test ROC AUC full 0.826, lean 0.824; `lean_features.txt` saved

SELF-CHECK

Q1

Forward selection kept both `support_tickets_12m` and `app_user`; `SelectKBest` with the F-test or with mutual information kept neither. What explains the difference?

A) Filters cannot score 0/1 columns such as `app_user`

B) The F-test needs scaled data and was given raw counts

C) A filter scores each column on its own; the wrapper judges each column next to the ones already chosen, so it sees that the two matter together

D) Forward selection always keeps more columns than `SelectKBest`

Q2

What does this print?

```
import pandas as pd
from sklearn.feature_selection import VarianceThreshold
X = pd.DataFrame({"flag": [0, 0, 0, 0, 1], "score": [1, 5, 2, 8, 4], "const":
print(VarianceThreshold(threshold=0.2).fit(X).get_feature_names_out())
```

Q3

An analyst has 300 customers and 2,000 survey columns. She runs `SelectKBest(f_classif, k=15)` on all 300 rows, then cross-validates a logistic regression on the 15 chosen columns and gets ROC AUC 0.84. What should she do?

A) Report 0.84: five folds are enough to trust it

B) Put the selector inside a pipeline and cross-validate the whole pipeline, expecting a lower number

C) Switch to mutual information, which detects more kinds of relationship

D) Use 10 folds instead of 5 to make the estimate more stable

Q4

Without a scaler, an L1 logistic regression on the churn columns keeps `avg_order_value` and drops `discount_share`, the second-best predictor. Why?

A) `discount_share` is redundant with spend

B) `avg_order_value` is the stronger predictor

C) L1 always keeps the column with more distinct values

D) `discount_share` runs from 0 to 1, so it needs a large weight, and the penalty charges for the size of each weight

Q5

Forward selection prints mean CV ROC AUC by number of columns: 1: 0.7215, 2: 0.8063, 3: 0.8165, 4: 0.8235, 5: 0.8289, 6: 0.8328, 7: 0.8336, 8: 0.8341, 9: 0.8358, 10: 0.8357. With a tolerance of 0.005, how many columns do you keep?

A) 6

B) 5

C) 9

D) 10

Q6

L1 set the weight of `avg_order_value` to zero. Which sentence belongs in your report?

A) Average order value has nothing to do with churn

B) Given spend and orders, average order value adds nothing new to this model

C) Average order value should be deleted from Pinecrest's database

D) Customers with a higher average order value churn less

JOURNAL

The noise experiment reported about 80% accuracy on data with no signal at all. In your own words, explain to a colleague why that happened and the one-line change that fixed it. Then list your six lean columns and, for each, write one sentence a Pinecrest marketer would understand about why it might matter, keeping in mind that the model describes patterns, not causes.

2.5 Imbalanced Classes

BY THE END YOU CAN

- Explain why accuracy fails on rare classes and judge a model with recall, precision and average precision against the right baseline
- Apply `class_weight="balanced"` and the imbalanced-learn samplers (random under- and oversampling, SMOTE) inside a leak-proof pipeline
- Show that weights and resampling mostly move the decision threshold, and explain what they do to predicted probabilities
- Choose a threshold from the cost of each kind of mistake, by formula and with `TunedThresholdClassifierCV`
- Check who carries the burden of false alarms before a model goes live

KEY IDEAS

- **The smoke detector.** Never beeping is right 364 days a year and worthless; beeping at toast catches every fire and gets unplugged. Where to sit between depends on what each mistake costs.
- **The accuracy paradox.** At 0.876% fraud, "all legitimate" is 99.1% accurate and catches nothing. Never compare imbalanced models by accuracy (or by `cross_val_score` without `scoring=`).
- **RANK, then DECIDE.** Judge the ranking with average precision against the base rate (about 0.009 here, not 0.5) and ROC AUC; judge decisions with recall, precision and cost.
- **Class weights and samplers mostly move the cut.** On fraud, `class_weight="balanced"`, under- and oversampling and SMOTE all gave about 1,900 flags, recall near 0.7 and no better ranking, and inflated the probabilities (mean 0.27 against a real 0.009).
- **Samplers go inside an imblearn pipeline.** A scikit-learn pipeline refuses them; resampling before CV faked an AP of 0.91 (honest: 0.49).
- **The threshold formula:** flag when $p \times$ (loss from a miss) > (cost of a review), so $p > 10 / 375 \approx 0.027$. It needs calibrated probabilities.
- **TunedThresholdClassifierCV** picks a threshold for any scorer with cross-validation on the training rows; `FixedThresholdClassifier` applies one you choose.
- **False alarms land on real customers.** Check whether one group carries more than its share.

WORKED EXAMPLE: A THRESHOLD FROM A CAPACITY

Lesson 2.5's "Choosing a Different Target" box: the fraud team can review only about 20 payments a day. Choose the cut that would have filled that queue in January to May, then use it on June, without looking at June first.

```
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

tx = pd.read_csv("data/transactions.csv")
cols = ["account_age_days", "amount", "hour", "items", "device_new", "billing_ship
        "country_mismatch", "card_attempts", "velocity_24h", "category"]
is_june = tx["timestamp"] >= "2025-06-01"
train, june = tx[~is_june], tx[is_june]
prep = ColumnTransformer([("num", StandardScaler(), cols[:-1]),
                          ("cat", OneHotEncoder(handle_unknown="ignore"), ["catego
model = make_pipeline(prep, LogisticRegression()).fit(train[cols], train["is_fraud
days_train = train["timestamp"].str[:10].nunique()
print(len(train), "training payments over", days_train, "days")
```

```
41738 training payments over 151 days
```

```
# the score that would have filled a 20-a-day queue in January to May
p_train = model.predict_proba(train[cols])[:, 1]
share = 20 * days_train / len(train)
threshold = np.quantile(p_train, 1 - share)
print(f"queue share {share:.2%}, threshold {threshold:.3f}")

flags = model.predict_proba(june[cols])[:, 1] >= threshold
caught = int(june.loc[flags, "is_fraud"].sum())
missed = int(june["is_fraud"].sum()) - caught
print(f"June: {flags.sum()} flagged ({flags.sum() / 30:.1f} a day), {caught} of {j
print(f"cost at $10 a review and $375 a missed fraud: ${10 * flags.sum() + 375 * m
```

```
queue share 7.24%, threshold 0.017
June: 596 flagged (19.9 a day), 47 of 64 frauds caught
cost at $10 a review and $375 a missed fraud: $12,335
```

The capacity threshold (0.017) is lower than the cost-tuned one from the lesson project (0.051), so June gets about 20 flags a day instead of 5.5. It catches 47 frauds instead of 41, but costs $12,335 against the tuned model's $10,285. Capacity is a limit, not a target: use it to rule thresholds out, and let cost choose among the rest.

GUIDED LAB

- **Lab A:** `labs/module-05/l2_5-imbalanced-classes/retention_threshold_starter.py`. A $15 retention offer against $60 lost per churner nobody contacts. Write the cost function first (TODO 1).
- **Lab B (pairs):** `labs/module-05/l2_5-imbalanced-classes/rebalance_tree_starter.py`. Does rebalancing rescue a decision tree? Predict the winner before you run TODO 2.
- Done when your numbers match the "Expected when done" lines.
- Finished early? `debug_imbalance.py`: two bugs that stop it, two that do not.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code. A wrong prediction you can explain afterward is worth more than a right guess.

P3

Predict the output, then run it. (Ten labels, two of them 1.)

```
import numpy as np
from sklearn.utils.class_weight import compute_class_weight
y = np.array([0] * 8 + [1] * 2)
print(compute_class_weight("balanced", classes=np.array([0, 1]), y=y))
```

P4

Predict the output, then run it. (Which order does ravel() use?)

```
from sklearn.metrics import confusion_matrix
y_true = [0, 0, 0, 1, 1]
y_pred = [0, 1, 0, 1, 0]
print(confusion_matrix(y_true, y_pred).ravel())
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

A fraud review queue for June (lesson page, "Project: A Fraud Review Queue for June"). In class: Steps 1 to 3. At home: Steps 4 to 6.

- ☐ Step 1: split by date. Check: 41,738 training payments (374 frauds), 8,262 in June (64)
- ☐ Step 2: June costs $24,000 reviewing nothing, $82,620 reviewing everything
- ☐ Step 3: CV AP plain 0.506, balanced 0.481, SMOTE 0.472
- ☐ Step 4: tuned threshold 0.051 (formula 0.027)
- ☐ Step 5: June once: 166 flags, 41 of 64 frauds, $10,285
- ☐ Step 6: `fraud_review_queue.csv`, sorted by probability, no `is_fraud` column

SELF-CHECK

Q7

What does this print?

```
import numpy as np
from sklearn.utils.class_weight import compute_class_weight
y = np.array([0] * 95 + [1] * 5)
print(compute_class_weight("balanced", classes=np.array([0, 1]), y=y).round(3)
```

Q8

A model's probabilities are calibrated. A manual review costs $8 and a missed fraud costs $200. Above which probability should a payment be flagged?

A) 0.5

B) 0.0088, the fraud rate

C) 0.4

D) 0.04

Q9

Resampling with SMOTE before cross-validation reported AP 0.912; with SMOTE inside an imblearn pipeline, 0.495. Which is honest, and why?

A) 0.495: resampling first put synthetic frauds, built from frauds in the training folds, into the validation folds, at a 50/50 balance production never has

B) 0.912: SMOTE gives the model more frauds to learn from, so it really is better

C) Neither: average precision cannot be used after resampling

D) 0.912, because the encoder was fitted on more rows

Q10

What does this print? (Review $10, missed fraud $375.)

```
from sklearn.metrics import confusion_matrix
y_true = [0, 0, 0, 0, 0, 0, 1, 1]
y_pred = [0, 1, 1, 0, 0, 0, 1, 0]
tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
print(10 * (tp + fp) + 375 * fn)
```

Q11

After `tuned = TunedThresholdClassifierCV(...).fit(X_train, y_train)`, what do `tuned.predict` and `tuned.predict_proba` return?

A) Both use the new threshold: `predict_proba` is rescaled so that 0.5 sits at the tuned threshold

B) `predict` still uses 0.5; only `predict_proba` changes

C) `predict` applies the tuned threshold; `predict_proba` returns the underlying model's probabilities unchanged

D) `predict_proba` is no longer available; only `predict` works

Q12

Cross-validating "all legitimate" and the plain fraud model without `scoring=` gives 0.9912 and 0.9941. What is wrong?

A) Nothing: the plain model is better, and the scores agree

B) The default score of a classifier is accuracy, which hides whether any fraud is found; pass `scoring="average_precision"` or a cost scorer

C) The folds were not stratified, so some had no fraud

D) The dummy should use `strategy="prior"`

◆ JOURNAL

Explain to a manager who has never heard of machine learning why "the model catches 69% of fraud" and "the model is 99% accurate" can both be misleading. Use your project's numbers: the queue size, the frauds caught and the cost against reviewing nothing. Then write down one group of honest customers who might carry an unfair share of the false alarms, and what you would do about it.

EXIT TICKET

Three questions, no notes.

E1

Why must `SelectKBest` live inside the pipeline when you cross-validate?

E2

A class-weighted fraud model has recall 0.69 where the plain model had 0.31, with the same AP. What changed, and what did it cost?

E3

A review costs $10 and a missed fraud $375. Which threshold, and on which model's probabilities may you use it?

NOTES

HOMEWORK CHECKLIST

- ☐ Finish the lean churn model (Steps 4 to 6) and the June review queue (Steps 4 to 6)
- ☐ Lesson 2.4 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Lesson 2.5 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Both Quick Quizzes and both journal entries
- ☐ "Before the Next Lesson" on both pages (noise experiment with `k=200`; my own two-cost problem; Step 3 with `scoring="roc_auc"`)
- ☐ Any lab TODO I did not finish; optional `debug_imbalance.py`
- ☐ Look back over my Module 1 and 2 notebooks: Checkpoint 1 opens Session 6 (open notebook, not graded)
- ☐ Bring to Session 6: a READY laptop, `pinecrest-ml` with `data/` and `labs/`, my notebooks from Lessons 1.1 to 2.5, `lean_features.txt`, this workbook and my journal

GLOSSARY

TERM	MEANING
redundant feature	A column the others already determine (spend = orders × average order value)
filter method	Scores each column against the target on its own (`SelectKBest`, `VarianceThreshold`)

TERM	MEANING
wrapper method	Tries subsets of columns with the real model (`RFECV`, `SequentialFeatureSelector`)
embedded method	Selects while training, such as an L1 penalty that sets weights to exactly zero
L1 penalty	A charge on the total size of the weights; `l1_ratio=1` in `LogisticRegression`, smaller `C` = more zeros
tolerance rule	Keep the smallest set whose CV score is within a small tolerance of the best
imbalanced classes	One class (the positive, minority class) is much rarer than the other
average precision (AP)	Area under the precision-recall curve; its baseline is the positive rate
class weight	A multiplier on each class's training mistakes; `"balanced"` = n / (2 × class count)
resampling	Changing the training rows: undersampling, oversampling, SMOTE (synthetic minority rows)
calibrated	Predicted probabilities that match real frequencies (a 0.2 means about one in five)
decision threshold	The probability above which the model says "positive"; 0.5 by default

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 6: Linear Models and Regularization

Your workbook for Session 6: the warm-up, Checkpoint 1, the key ideas of Lessons 3.1 and 3.2, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 3.1–3.2 | Session 6 · 3 hours | Name: _____

SESSION GOALS

- Show what you can do with Modules 1 and 2 in Checkpoint 1: find a leak, rebuild a model honestly, decide from costs.
- Fit and read linear and logistic regressions: coefficients in units, in percent and as odds ratios.
- Recognize underfitting and overfitting from training and test errors, and explain bias and variance.
- Use Ridge and Lasso in a pipeline with scaling, and choose their penalty by cross-validation.

WARM-UP: SELECTION AND IMBALANCE

W1

RFECV's curve is 0.8328 at 6 columns and 0.8333 at 11; the folds disagree by about 0.02. How many columns do you keep, and why?

W2

Review $10, missed fraud $375. What does this print?

```
import numpy as np
p = np.array([0.02, 0.04, 0.3, 0.6])
print((p >= 10 / 375).astype(int))
```

W3

Where must SMOTE sit when you cross-validate, and what happens otherwise?

W4

Why must an L1 model be scaled before it selects columns?

W5

A class-weighted model's probabilities average 0.27 when the real fraud rate is 0.009. What does that change?

CHECKPOINT 1: AUDIT AND FIX A CHURN MODEL

THE TASK (30 MINUTES, OPEN NOTEBOOK, NOT GRADED)

Pinecrest's marketing team will email a win-back voucher to the customers a churn model flags. An email with a voucher costs **$5**; a churner nobody reaches costs about **$50** of margin. A colleague's notebook (Part A of `labs/module-06/checkpoint-1/checkpoint1_starter.py`) reports about 95% accuracy. **Plan** what you will check; **audit** the colleague's code (one line per problem); **rebuild** an honest model in a pipeline, cross-validated against a baseline with a metric suited to a rare class, and choose who gets the email from the prices, on the training rows only; look at the test set **once**; **report** three sentences for the marketing lead.

- PLAN (3 min) · AUDIT (5 min) · REBUILD (15 min) · REPORT (5 min). Write the report even if the code is unfinished.
- You will be looking for: a leak, preprocessing that saw the test rows, a missing baseline, a decision that ignores the prices.
- Afterward you get the rubric. Its five rows: leakage audit · pipeline and preprocessing · honest evaluation · decision from costs · communication.

LESSONS

3.1 Linear and Logistic Regression

BY THE END YOU CAN

- Explain what least squares does and fit `LinearRegression`, reading `coef_` and `intercept_` in the units of the data
- Interpret a coefficient in a multiple regression as an effect "holding the other features constant", and spot when that reading breaks down
- Use a log target with `TransformedTargetRegressor` when errors grow with the size of the prediction
- Describe how logistic regression turns a weighted sum into a probability, and read its coefficients as log-odds and odds ratios
- Fit a multiclass logistic regression and know which settings are current (`C`, `l1_ratio`) and which are gone

KEY IDEAS

- **The ruler on the scatter plot.** Least squares picks the line with the smallest sum of squared residuals; for one feature, slope = covariance / variance.
- **Read coefficients in units.** -0.76 snow items per °F; an intercept can be meaningless (0 °F never happens).
- **Holding the others constant.** A coefficient's meaning depends on what else is in the model: the price effect went from -0.53 to -1.61.
- **Lines extrapolate without limits** and give each feature one slope for everyone.
- **A fanning residual plot means percent effects:** model `log1p(y)` with `TransformedTargetRegressor`; read `exp(coef) - 1` as a percentage.
- **Logistic regression:** $z = b_0 + b \cdot x$ is the log-odds; $p = 1 / (1 + e^{-z})$; `exp(coef)` is an odds ratio; `decision_function > 0` is the same as $p > 0.5$.
- **Current API:** multiclass uses softmax automatically (`multi_class` is gone); the penalty is `C` and `l1_ratio` (`penalty` is deprecated).
- **Coefficients describe the model, not causes;** twin columns split credit arbitrarily.

WORKED EXAMPLE: THREE CUSTOMERS BY HAND

Two features, no scaler, so each coefficient is "per one unit": being an app user (0 or 1) and one more support ticket. Compute three customers' probabilities by hand.

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

customers = pd.read_csv("data/customers_churn.csv")
train, test = train_test_split(customers, test_size=0.25, stratify=customers["chur
cols = ["app_user", "support_tickets_12m"]
m = LogisticRegression().fit(train[cols], train["churned"])
b0 = m.intercept_[0]
b_app, b_tix = m.coef_[0]
print(f"z = {b0:.3f} {b_app:+.3f} * app_user {b_tix:+.3f} * tickets")
```

```
z = -1.630 -0.745 * app_user +0.258 * tickets
```

```
for app, tix in [(0, 0), (0, 4), (1, 4)]:
    z = b0 + b_app * app + b_tix * tix
    print(f"app_user={app}, tickets={tix}: z = {z:+.3f}, p = {1 / (1 + np.exp(-z))
real = train[train["support_tickets_12m"] >= 4].groupby("app_user")["churned"].mea
print("real churn with 4+ tickets, by app_user:", real.round(3).to_dict())
```

```
app_user=0, tickets=0: z = -1.630, p = 0.164
app_user=0, tickets=4: z = -0.599, p = 0.354
app_user=1, tickets=4: z = -1.344, p = 0.207
real churn with 4+ tickets, by app_user: {0: 0.464, 1: 0.038}
```

The by-hand sigmoid gives the model's probabilities. But compare the last line: among customers with 4 or more tickets, 46.4% of non-app users churned and only 3.8% of app users. The model says 35% and 21%: with one slope per feature it cannot say "tickets matter only without the app". The interaction feature from Lesson 2.3 (`tickets_no_app`) is what lets a linear model say it.

GUIDED LAB

- **Lab A:** `labs/module-06/l3_1-linear-logistic/daily_sales_line_starter.py`. Daily store sales: baselines, a growth line by hand, coefficients in dollars, a twin column, a log target.
- **Lab B (pairs):** `labs/module-06/l3_1-linear-logistic/fraud_odds_starter.py`. Odds ratios an analyst can read: per flag, per $100.
- Done when your numbers match the "Expected when done" lines.

PREDICT, THEN RUN

Write your prediction first, then run the code.

P1

Predict the output, then run it. (Three points; where is the line at x = 4?)

```
from sklearn.linear_model import LinearRegression
line = LinearRegression().fit([[0], [1], [2]], [1, 1, 4])
print(line.coef_.round(2), round(line.intercept_, 2), line.predict([[4]]).rou
```

P2

Predict the output, then run it. (Odds of 3 to 1.)

```
import numpy as np
z = np.log(3)            # log-odds: odds of 3 to 1
print(round(1 / (1 + np.exp(-z)), 2))
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

What does a discount really sell? In class: Steps 1 to 3. At home: Steps 4 to 6.

- ☐ Step 1: log columns and crossed categories
- ☐ Step 2: naive elasticity -0.53
- ☐ Step 3: within-product -1.50
- ☐ Step 4: full model -1.606; a 10% cut sells +18.4%; promo +26.2%
- ☐ Step 5: 2024 only, 2025 MAE 8.27 (baseline 13.42)
- ☐ Step 6: the tent what-if: +45% units, +16% revenue, assumptions named

SELF-CHECK

Q1

What does this print?

```
from sklearn.linear_model import LinearRegression
line = LinearRegression().fit([[0], [1], [2]], [5, 8, 11])
print(line.predict([[10]]))
```

Q2

A demand model trained on `log1p(units_sold)` gives `promo` a coefficient of 0.420. What does that say?

A) A promo week sells 0.42 more units

B) A promo week sells about 52% more, for every product, whatever its size

C) A promo week sells 42% more

D) Promotions explain 42% of the variation in sales

Q3

An unscaled logistic regression gives the 0/1 flag `device_new` a coefficient of 0.69. Which sentence is right?

A) A new device adds 0.69 to the probability of fraud

B) A new device makes fraud 69% more likely

C) A new device causes fraud

D) A new device multiplies the odds of fraud by about 2, holding the other features constant

Q4

The lesson project's naive price elasticity was -0.53; adding `product_id` made it -1.50. Why did it change so much?

A) The naive slope compared different products (cheap socks sell more units than tents), so price stood in for product type

B) Adding a column always makes coefficients larger

C) The log of the price was taken twice

D) The second model was fitted on fewer rows

Q5

A logistic regression's decision_function returns -1.0986 for one customer. What does this print?

```
import numpy as np
z = -1.0986
print(round(1 / (1 + np.exp(-z)), 2))
```

Q6

Someone fits `LogisticRegression()` with the whole-number target `units_sold`. What happens?

A) `ValueError: Unknown label type: continuous`

B) It fits a regression line through the counts

C) No error: every distinct count becomes its own class, which is wrong and slow

D) It predicts probabilities of selling more than the mean

JOURNAL

Your project produced three different "effects of price": -0.53, -1.50 and -1.61. Write a short note to a merchandising manager explaining why the first two are misleading, in plain words with no math. Then

write one sentence each on what a logistic regression coefficient of +0.67 for `discount_share` does and does not tell Pinecrest.

3.2 Regularization and the Bias-Variance Trade-off

BY THE END YOU CAN

- Explain underfitting and overfitting in terms of bias and variance, and recognize each from training and test errors
- Add polynomial and interaction features with `PolynomialFeatures` and predict how many columns they create
- Fit `Ridge`, `Lasso` and `ElasticNet` in a pipeline with scaling, and describe how their penalties differ
- Choose the penalty strength `alpha` by cross-validation on the training data only, with `RidgeCV` and `LassoCV`
- Build a regularized demand forecast with interactions and judge it against baselines on a later year

KEY IDEAS

- **Three students:** one rule for everything (bias), every answer memorized (variance), real patterns learned (the goal).
- **The diagnostic:** high training and test error = too simple; low training and much higher test error = too flexible.
- **Expected test error = bias^2 + variance + noise.** More flexibility trades bias for variance: the U.
- **PolynomialFeatures** adds powers and products; the column count explodes with the degree.
- **Ridge** charges for squared weights and shrinks them all; **Lasso** charges for absolute weights and zeros many; **ElasticNet** mixes both.
- **Penalties need scaled features,** inside the pipeline.
- **Choose alpha by cross-validation** on the training rows (`RidgeCV`, `LassoCV`), searched on a log scale; test once.
- **More data is also a regularizer.** With few rows and many columns the penalty decides everything.

WORKED EXAMPLE: A RIDGE PATH

Watch Ridge pull the weights of a Snow demand model toward zero as alpha grows (features scaled, log target).

```
import numpy as np
import pandas as pd
from sklearn.linear_model import Ridge
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

demand = pd.read_csv("data/weekly_demand.csv")
snow = demand[(demand["year"] == 2024) & (demand["category"] == "Snow")]
cols = ["avg_high_temp_f", "discount_pct", "promo", "holiday_week"]
rows = {}
for alpha in [0.1, 100, 1_000, 10_000, 100_000]:
    m = make_pipeline(StandardScaler(), Ridge(alpha=alpha)).fit(snow[cols], np.log
    rows[alpha] = m[-1].coef_
print(pd.DataFrame(rows, index=cols).round(3))
```

```
                  0.1       100.0     1000.0    10000.0   100000.0
avg_high_temp_f    -1.285    -1.012    -0.358    -0.049    -0.005
discount_pct        0.143     0.058    -0.028    -0.008    -0.001
promo               0.058     0.039    -0.016    -0.006    -0.001
holiday_week        0.151     0.105     0.022     0.002     0.000
```

Temperature carries the model, so it survives the longest; the small weights for discounts and promotions are pulled to zero quickly (and even change sign on the way). At alpha = 100,000 every weight is close to 0 and the model predicts almost the average week: maximum bias. None is ever exactly 0, which is the difference from Lasso.

GUIDED LAB

- **Lab A:** `labs/module-06/l3_2-regularization/churn_curves_starter.py`. Polynomial churn features on a 1,000-customer pilot, a validation curve over C, then all customers.
- **Lab B (pairs):** `labs/module-06/l3_2-regularization/variance_without_truth_starter.py`. Measure variance on real data by refitting on 30 pilot samples.
- Done when your numbers match the "Expected when done" lines.

PREDICT, THEN RUN

Write your prediction first, then run the code.

P3

Predict the output, then run it. (Four columns, products of pairs only.)

```
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
pf = PolynomialFeatures(degree=2, interaction_only=True, include_bias=False).f
print(pf.n_output_features_)
```

P4

Predict the output, then run it. (The same line, with and without a penalty.)

```
from sklearn.linear_model import LinearRegression, Ridge
X, y = [[0], [1], [2], [3]], [10, 12, 14, 16]
print(LinearRegression().fit(X, y).coef_, Ridge(alpha=10).fit(X, y).coef_.roun
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The interaction forecast. In class: Steps 1 and 2. At home: Steps 3 to 6.

- ☐ Step 1: baselines 17.13 and 13.42
- ☐ Step 2: Ridge, main effects: 10.62
- ☐ Step 3: 489 columns
- ☐ Step 4: RidgeCV chooses alpha 1.0
- ☐ Step 5: 2025 MAE 7.92, R^2 0.754
- ☐ Step 6: the biggest interactions; `demand_forecast_2025.csv` (4,896 rows)

SELF-CHECK

Q7

An interaction model trained on a 400-row pilot has training MAE 3.4 and test MAE 42.2. Which change is most likely to help?

A) Remove the penalty so the model can fit better

B) Add degree-3 terms to capture more patterns

C) Increase the Ridge penalty (alpha), chosen by cross-validation

D) Judge it on the training rows, where it does well

Q8

How many columns does this report?

```
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
print(PolynomialFeatures(degree=2, include_bias=False).fit(np.zeros((1, 4))).n
```

Q9

Snow sales get the temperature twice, in °F and in °C. Ridge gives weights [-0.607, -0.607]; Lasso gives [-1.164, 0]. What follows?

A) Ridge shares the effect between the twins; Lasso keeps one twin and zeros the other, somewhat arbitrarily

B) Lasso is wrong and Ridge is right

C) The Celsius column does not matter, so temperature in °C has no effect on sales

D) The penalties need more scaling

Q10

Without a scaler, Ridge(alpha=100) fits Snow sales with the price in dollars (R^2 0.770) and with the price in thousands of dollars (R^2 0.752). Why do the fits differ?

A) Prices in thousands are rounded

B) Ridge cannot use prices below 1

C) R^2 changes whenever the units change

D) In thousands the price needs a weight 1,000 times larger, and the penalty shrinks it hard

Q11

What does this print?

```
from sklearn.linear_model import Ridge
m = Ridge(alpha=1e6).fit([[0], [1], [2], [3]], [10, 12, 14, 16])
print(m.coef_.round(3), round(m.intercept_, 1))
```

Q12

RidgeCV chose alpha = 3.16 on the 400-row pilot but alpha = 1.0 on the full 5,088 rows of 2024. Why less penalty with more data?

A) Cross-validation is unreliable on small data, so the first number is wrong

B) More rows reduce variance on their own, so less penalty is needed to control it

C) Ridge always prefers alpha = 1

D) The full year has fewer columns

JOURNAL

Describe the three students and the practice exam in your own words, then match each to a model you trained today. Finally, write one sentence a Pinecrest planner would understand that explains why the forecaster's alpha was chosen by cross-validation and not by the 2025 results.

EXIT TICKET

E1

A churn model gives `discount_share` (scaled) a coefficient of 0.67. Say what it means in one sentence, and one thing it does not mean.

E2

Training error tiny, test error large: name the problem and two fixes.

E3

Why must features be scaled before Ridge or Lasso?

NOTES

HOMEWORK CHECKLIST

- ☐ Finish the 3.1 project (Steps 4 to 6) and the 3.2 project (Steps 3 to 6)
- ☐ Lesson 3.1 and 3.2 Hands-on Exercises 1 to 3 (Exercise 4 of each is a stretch)
- ☐ Both Quick Quizzes and both journal entries
- ☐ Finish Checkpoint 1 if needed and compare it with the rubric
- ☐ "Before the Next Lesson" on both pages
- ☐ Bring to Session 7: a READY laptop, `pinecrest-ml` with `data/` and `labs/`, my 3.2 project notebook, my checkpoint file, this workbook and my journal

GLOSSARY

TERM	MEANING
least squares	The rule that picks the line with the smallest sum of squared residuals
residual	Actual minus predicted, for one row
log target	Modeling `log1p(y)` so effects act in percent; `TransformedTargetRegressor` converts back
log-odds	z = log(p / (1 − p)), the weighted sum inside logistic regression
odds ratio	`exp(coef)`: the factor by which the odds change for a one-unit increase
collinearity	Columns that (almost) repeat each other, so their coefficients split credit arbitrarily
bias / variance	How wrong the average fit is / how much the fit changes from sample to sample
regularization	A penalty on large weights that trades a little bias for less variance
Ridge / Lasso / ElasticNet	Squared-weight penalty / absolute-weight penalty (zeros) / a mix
alpha, C	Penalty strength in Ridge and Lasso; `C` in LogisticRegression is its inverse (small C, strong penalty)

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 7: Nearest Neighbors and Naive Bayes

Your workbook for Session 7: the warm-up, the key ideas of Lessons 4.1 and 4.2, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 4.1–4.2 | Session 7 · 3 hours | Name: _____

SESSION GOALS

- Compute distances and see why scaling decides who the neighbors are.
- Choose k by cross-validation and read small and large k as variance and bias.
- Use lookalikes to explain a prediction, with a privacy caveat.
- Compute a posterior with Bayes' rule and explain the naive assumption.
- Classify text with CountVectorizer and Naive Bayes, and tell ranking from calibrated probabilities.

WARM-UP: LINEAR MODELS AND REGULARIZATION

W1

A log-target demand model gives `holiday_week` a coefficient of 0.405. In words?

W2

A logistic regression flag has coefficient 0.69. What does this print, and what does it mean?

```
import numpy as np
print(round(np.exp(0.69), 2))
```

W3

Degree 15 has training MSE 0.016 and test MSE 0.187 (noise floor 0.09). Which problem, and which dial helps?

W4

Temperature in °F and °C: Ridge gives [-0.607, -0.607], Lasso [-1.164, 0]. What does Lasso's zero mean?

W5

Why must features be scaled before Ridge or Lasso?

LESSONS

4.1 k-Nearest Neighbors

BY THE END YOU CAN

- Explain how k-nearest neighbors classifies a row, and compute Euclidean and Manhattan distances by hand
- Show why unscaled features wreck distances, and fix it with a scaler inside a pipeline
- Choose `k` with cross-validation and connect small and large `k` to variance and bias
- Recognize the curse of dimensionality and why irrelevant features hurt kNN more than a linear model
- Use `kneighbors` to show the lookalike customers behind a prediction

KEY IDEAS

- **The agent's comps.** Predict from the k most similar past rows: a vote share for classification, an average for regression.
- **Distance defines similarity.** Euclidean (`p=2`) or Manhattan (`p=1`); without scaling, the column with the biggest numbers decides.
- **Scaling decides everything:** churn AUC 0.774 unscaled, 0.836 scaled, same k.
- **k is the dial.** k = 1: variance (training AUC 1.000); huge k: bias. Choose k by cross-validation; prefer the middle of a flat plateau.
- **Lookalikes explain predictions,** but they are other customers' data: summarize, do not print.
- **The curse of dimensionality.** Irrelevant columns blur every distance; for kNN, feature selection is part of the model.
- **Lazy learner:** fit stores, predict searches. Fast to train, slow to predict, ships with its training data.

WORKED EXAMPLE: SIMILAR PRODUCTS

Find the three products most like a tent by price and launch year, with and without scaling. `NearestNeighbors` is kNN without the prediction: it only finds the closest rows.

```
import numpy as np
import pandas as pd
from sklearn.neighbors import NearestNeighbors
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

products = pd.read_csv("data/products_ml.csv")
cols = ["price", "launch_year"]
tent = products[products["category"] == "Tents"].iloc[[0]]
print(tent[["name", "price", "launch_year"]].to_string(index=False))
```

```
                          name  price  launch_year
Juniper Vane Drift 2-Person Tent 398.95         2024
```

```
for name, prep in [("unscaled", None), ("scaled", StandardScaler())]:
    X = products[cols] if prep is None else prep.fit_transform(products[cols])
    nn = NearestNeighbors(n_neighbors=4).fit(X)
    query = tent[cols] if prep is None else prep.transform(tent[cols])
    _, idx = nn.kneighbors(query)
    near = products.iloc[idx[0][1:]]
    print(name, list(zip(near["price"], near["launch_year"])))
```

```
unscaled [(409.95, 2021), (371.95, 2019), (426.95, 2025)]
scaled [(442.95, 2024), (426.95, 2025), (317.95, 2024)]
```

Unscaled, prices span hundreds of dollars and years only six, so "similar" means "similar price": the lookalikes come from 2019, 2021 and 2025. Scaled, a year counts as much as a standard deviation of price, and two of the three are 2024 products. Neither is "right": scaling is a modeling choice that defines similarity.

GUIDED LAB

- **Lab A:** `labs/module-07/l4_1-knn/fraud_neighbors_starter.py`. kNN on fraud: who decides the unscaled distance, scaled against unscaled, k by cross-validated AP.
- **Lab B (pairs):** `labs/module-07/l4_1-knn/lookalike_days_starter.py`. Forecast Boise's days from lookalike days.
- Finished early? `debug_knn.py`: two loud bugs, two silent ones.

PREDICT, THEN RUN

Write your prediction first, then run the code.

P1

Predict the output, then run it. (Which two training points are nearest 3?)

```
from sklearn.neighbors import KNeighborsRegressor
reg = KNeighborsRegressor(n_neighbors=2).fit([[1], [2], [4], [8]], [10, 20, 40
print(reg.predict([[3]]))
```

P2

Predict the output, then run it. (A 3-4-5 triangle.)

```
from sklearn.metrics import pairwise_distances
pts = [[0, 0], [3, 4]]
print(pairwise_distances(pts)[0, 1], pairwise_distances(pts, metric="manhattan
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Lookalike churn scoring. In class: Steps 1 to 3. At home: Steps 4 to 6.

- ☐ Step 1: 4,500 / 1,500, test churn rate 0.165
- ☐ Step 2: baseline 0.500, logistic regression 0.823
- ☐ Step 3: lean 6, k = 51, distance weights: CV AUC 0.8552
- ☐ Step 4: test AUC 0.855; 31 churners in the top 50
- ☐ Step 5: a reason per customer from a summary of lookalikes
- ☐ Step 6: `lookalike_shortlist.csv` (50 rows, no outcome)

SELF-CHECK

Q1

What does this print?

```
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=3).fit([[0], [1], [2], [6], [7]], [1, 1
print(knn.predict_proba([[1.2]]).round(3))
```

Q2

As `n_neighbors` grows from 1 to 501 on the churn data, what happens?

A) Variance falls and bias rises: predictions drift toward the overall churn rate

B) Variance rises: more neighbors means more noise in each vote

C) Nothing changes once the features are scaled

D) The model trains more slowly, because it learns more weights

Q3

Unscaled kNN on fraud scored AP 0.056 (scaled: 0.397). The first two training payments' squared distance was 99.7% account age. What went wrong?

A) kNN cannot handle rare classes

B) 51 neighbors is too many for fraud

C) Account age runs up to 7,030 days, so it alone decided who the neighbors were; the flags that signal fraud had no say

D) Average precision does not work for kNN

Q4

Fifty columns of random noise lower kNN's churn AUC from 0.842 to 0.755 but logistic regression's only from 0.823 to 0.816. Why?

A) kNN needs more training iterations when there are more columns

B) kNN gives every scaled column an equal say in the distance; logistic regression learns near-zero weights for useless columns

C) Logistic regression scales its own inputs

D) The noise columns were not scaled for kNN

Q5

What does this print?

```
from sklearn.metrics import pairwise_distances
print(pairwise_distances([[1, 2, 3]], [[4, 0, 3]], metric="manhattan"))
```

Q6

Pinecrest wants to put the lookalike churn model into a nightly scoring job. What must ship with the fitted kNN model?

A) Only the scaler's means and standard deviations

B) One weight per feature, like logistic regression

C) Nothing: kNN predicts from the new customer alone

D) The scaler and every training customer, because each prediction searches them

JOURNAL

Explain the real-estate agent's comps in your own words, and name the two decisions that make or break the estimate. Then write down the unscaled and scaled AUC from this lesson and explain the difference to

someone who has never heard of standard deviations. Finally: kNN beat logistic regression on churn but lost badly once noise columns were added. What does that tell you about when to reach for each?

4.2 Naive Bayes

BY THE END YOU CAN

- Compute a posterior probability by hand with Bayes' rule, starting from counts in a table
- Explain the "naive" independence assumption and show what it costs when features repeat each other
- Turn review text into word counts with `CountVectorizer` and classify it with `MultinomialNB` inside a pipeline
- Explain what the smoothing parameter `alpha` prevents, and choose between the Multinomial, Bernoulli, Complement and Gaussian variants
- Tell a model that ranks well apart from one whose probabilities you can take at face value

KEY IDEAS

- **The store manager.** A prior, clues that multiply the odds, a posterior.
- **Bayes' rule:** P(class | clue) = P(clue | class) × P(class) / P(clue). Odds form: posterior odds = prior odds × likelihood ratio.
- **The naive assumption:** clues are independent within a class, so their probabilities multiply. Often false; rankings still decent.
- **Bag of words:** `CountVectorizer` makes one sparse column per word; word order is lost ("would not buy again").
- **Smoothing** (`alpha`) adds a pretend count so no unseen word can veto the rest. Keep it above 0.
- **Variants:** Multinomial or Complement for counts, Bernoulli for presence, Gaussian for measurements. Never scale count inputs.
- **Duplicated features are counted twice:** removing them can help Naive Bayes.
- **Good ranking, overconfident probabilities:** rank with it; recalibrate before quoting it.

WORKED EXAMPLE: A TWO-WORD CLUE

Bayes' rule with a two-word clue. Is "waterproof" good news?

```
import pandas as pd
reviews = pd.read_csv("data/reviews.csv")
labeled = reviews[reviews["rating"] != 3]
neg = labeled["rating"] <= 2
for pattern in [r"\bwaterproof\b", r"\bnot waterproof\b"]:
    has = labeled["text"].str.contains(pattern, case=False)
    print(f"{pattern:22s} reviews {has.sum():4d}   P(negative | it) {neg[has].mean
print(f"prior P(negative) {neg.mean():.3f}")
```

```
\bwaterproof\b         reviews  145   P(negative | it) 0.310
\bnot waterproof\b     reviews   58   P(negative | it) 0.603
prior P(negative) 0.255
```

Before reading, a review is negative with probability 0.255. A review that mentions "waterproof" is negative 31% of the time; one that says "not waterproof" 60%. Most of the word's bad news comes from the pair, which a bag of single words cannot see: that is why the lesson project tries word pairs (`ngram_range=(1, 2)`).

GUIDED LAB

- **Lab A:** `labs/module-07/l4_2-naive-bayes/fraud_flags_bayes_starter.py` . Bayes' rule on fraud flags; the naive product against direct counts.
- **Lab B (pairs):** `labs/module-07/l4_2-naive-bayes/review_category_starter.py` . Which of ten categories is a review about?
- Done when your numbers match the "Expected when done" lines.

PREDICT, THEN RUN

Write your prediction first, then run the code.

P3

Predict the output, then run it. (binary=True records presence.)

```
from sklearn.feature_extraction.text import CountVectorizer
v = CountVectorizer(binary=True)
print(v.fit_transform(["warm warm boots", "boots"]).toarray())
```

P4

Predict the output, then run it. (Prior odds times the likelihood ratio.)

```
prior, ratio = 0.25, 3            # P(negative) and P(word | neg) / P(word | p
odds = prior / (1 - prior) * ratio
print(round(odds / (1 + odds), 2))
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Review triage for customer care. In class: Steps 1 to 3. At home: Steps 4 to 6.

- ☐ Step 1: 2,321 / 774 / 905; first test ID R02194
- ☐ Step 2: CV AUC 0.899 (words) → 0.912 (word pairs)
- ☐ Step 3: alpha 0.3
- ☐ Step 4: accuracy 0.868, AUC 0.934, negative recall 0.843
- ☐ Step 5: 457 of 905 three-star reviews read as negative
- ☐ Step 6: 60 possible mismatches; both CSV files saved

SELF-CHECK

Q7

20% of reviews are negative. The word "flimsy" appears in 10% of negative reviews and 1% of positive ones. What is P(negative | "flimsy")?

A) 0.10

B) 0.20, the prior does not change

C) 0.50

D) About 0.71

Q8

What does this print?

```
from sklearn.feature_extraction.text import CountVectorizer
v = CountVectorizer()
print(v.fit_transform(["not bad not", "bad"]).toarray())
```

Q9

With `MultinomialNB(alpha=1e-10)` an angry review that says "leaked", "fell apart" and "returned" but also "love" comes out 100% positive. Why?

A) "love" never appeared in a negative training review, so with almost no smoothing its probability in that class is about 0, and one zero factor vetoes everything else

B) "love" is the strongest word in the vocabulary

C) The model ignores negative words

D) Small alpha makes the model prefer the majority class

Q10

`make_pipeline(StandardScaler(), MultinomialNB())` on the churn columns raises `ValueError: Negative values in data passed to MultinomialNB (input X).` What should you use?

A) A MinMaxScaler, so the values are between 0 and 1

B) ComplementNB, which accepts negative counts

C) GaussianNB, unscaled: it models continuous measurements with a bell curve per class

D) Take absolute values before MultinomialNB

Q11

GaussianNB on 10 churn columns scores AUC 0.778; on 6 (without spend, average order value and the two returns columns) 0.813. Why does dropping columns help?

A) Six columns train faster

B) The dropped columns repeat information already present, and Naive Bayes counts each one as a new, independent clue

C) The dropped columns had missing values

D) GaussianNB can only use six columns

Q12

What does this print?

```
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
print("not" in ENGLISH_STOP_WORDS, "boots" in ENGLISH_STOP_WORDS)
```

JOURNAL

Explain the store manager's prior and clues using one review from your project's mismatch list. Then describe, in your own words, why a model can rank customers well (high AUC) while its probabilities are too confident. Which of those two properties would matter more for the customer-care team, and which for a finance forecast?

EXIT TICKET

E1

Why does kNN need scaled features, and why does MultinomialNB refuse them?

E2

k = 1 gives training AUC 1.000 and test AUC 0.645. Name the problem and the fix.

E3

Prior P(negative) = 0.25; a word appears in 8% of negative and 2% of positive reviews. What is the posterior?

NOTES

HOMEWORK CHECKLIST

- ☐ Finish the 4.1 project (Steps 4 to 6) and the 4.2 project (Steps 4 to 6)
- ☐ Lesson 4.1 and 4.2 Hands-on Exercises 1 to 3 (Exercise 4 of each is a stretch)
- ☐ Both Quick Quizzes and both journal entries

- ☐ "Before the Next Lesson" on both pages
- ☐ Any lab TODO I did not finish; optional `debug_knn.py`
- ☐ Bring to Session 8: a READY laptop, `pinecrest-ml` with `data/` and `labs/`, my 4.1 and 4.2 notebooks, `lookalike_shortlist.csv`, this workbook and my journal

GLOSSARY

TERM	MEANING
k-nearest neighbors	Predicts from the k most similar training rows
Euclidean / Manhattan distance	Straight-line distance / sum of absolute differences
lazy learner	A model whose fit only stores the data; the work happens at predict time
curse of dimensionality	In many dimensions all points are about equally far apart
prior / posterior	Belief before / after seeing the evidence
likelihood ratio	P(clue | class A) / P(clue | class B): how much a clue multiplies the odds
naive assumption	Features are independent within each class
bag of words	One column per word, counts per document, word order ignored
smoothing (alpha)	A pretend count added to every word so no probability is zero
calibration	Whether predicted probabilities match real frequencies

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 8: Support Vector Machines and Decision Trees

Your workbook for Session 8: a warm-up on Session 7, the key ideas of Lessons 4.3 and 5.1, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 4.3–5.1 | Session 8 · 3 hours | Name: _____

SESSION GOALS

- Picture an SVM as the widest street between two classes, and predict what the C and gamma dials do to it.
- Explain the kernel trick with the ring example, and build a scaled, tuned, class-weighted SVM pipeline.
- Read a decision tree as rules, compute a node's Gini impurity, and say where a tree's probabilities come from.
- Choose a tree's size by cross-validation, and read its importances without over-reading them.
- Know when to use each: readable rules, a ranked list, or neither.

WARM-UP: NEAREST NEIGHBORS AND NAIVE BAYES

Five questions on Session 7. Two minutes alone, then compare with a neighbor.

W1

Lesson 4.1: a kNN churn model scores a perfect 1.000 on its training data with k=1. Why, and what do you do about it?

W2

Two unscaled features: spend differs by $300 and days since last order by 120. What share of the squared distance comes from spend? Predict, then check.

```
import numpy as np
spend, days = 300, 120
print(round(spend ** 2 / (spend ** 2 + days ** 2), 2))
```

W3

Lesson 4.2: a word appears in 6% of negative reviews and 0.5% of positive ones; 25% of reviews are negative. After seeing the word, about how likely is the review to be negative?

W4

Why does MultinomialNB need alpha above 0?

W5

Your Naive Bayes model ranks customers well (AUC 0.81) but its probabilities pile up near 0 and 1. Can you quote them to the care team?

LESSONS

4.3 Support Vector Machines

BY THE END YOU CAN

- Explain the maximum-margin idea and find the support vectors of a fitted model
- Predict how changing C and gamma changes the margin, the number of support vectors and the risk of overfitting
- Explain the kernel trick and use the RBF kernel to learn a curved boundary
- Build a scaled SVC pipeline, rank with `decision_function`, handle imbalance with `class_weight`, and tune C and gamma with cross-validation
- Decide when an SVM is a sensible choice and when its training time rules it out

KEY IDEAS

- **The widest street.** An SVM picks the boundary with the widest margin. Only the support vectors (points on or inside the street, or misclassified) shape it.
- **C prices violations.** Small C: wide street, many support vectors, simpler model. Large C: narrow street that fits training points harder. C is the inverse of regularization strength.
- **The kernel trick.** Data tangled in its own features can become separable after a lift into more features (the ring plus $x^2 + y^2$). A kernel computes similarities as if the lift had happened, without building the features.
- **gamma (RBF kernel).** How far each point's influence reaches: similarity = exp(-gamma × squared distance). Too large memorizes noise; tune C and gamma together by cross-validation (keys `svc__C`, `svc__gamma`).
- **Always scale, inside a pipeline.** Kernels and margins are distances.
- **No probabilities.** Rank with `decision_function`; calibrate with `CalibratedClassifierCV(SVC(), ensemble=False)` when you need probabilities.
- **Imbalance:** `class_weight="balanced"`; without it an SVM can ignore the minority class (accuracy = baseline, nobody flagged).
- **Training time** grows roughly with the square of the rows: kernel SVM up to about 10,000 rows, LinearSVC beyond.

WORKED EXAMPLE: WHERE DO THREE NEW CUSTOMERS STAND?

Where do three new customers stand relative to the street? An SVM on just two scaled features, recency and discount share, so the street is a line you could draw.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

customers = pd.read_csv("data/customers_churn.csv")
pair = ["days_since_last_order", "discount_share"]
X_train, X_test, y_train, y_test = train_test_split(
    customers[pair], customers["churned"], test_size=0.25, stratify=customers["chu
street = make_pipeline(StandardScaler(), SVC(kernel="linear", class_weight="balanc
print("support vectors per class:", street[-1].n_support_)
print("weights (scaled features):", street[-1].coef_[0].round(2))
```

```
support vectors per class: [2287  454]
weights (scaled features): [0.87 0.7 ]
```

With `class_weight="balanced"` and heavily overlapping classes, 2,741 of 4,500 customers sit on or inside the street. Both weights are positive: more days since the last order and more discount use push toward churn.

```
new = pd.DataFrame({"days_since_last_order": [20, 150, 300], "discount_share": [0.
print(street.decision_function(new).round(2))
print(street.predict(new))
```

```
[-1.59  1.09  3.37]
[0 1 1]
```

The scores are signed distances in street widths. The first customer (ordered 20 days ago, rarely uses discounts) is 1.59 on the "stays" side, beyond the curb. The second (150 days, 40% discounts) is inside the street on the churn side: predicted to churn, but not confidently. The third is far out on the churn side. Sort by these scores and you have a ranking; read 3.37 as a probability and you have a mistake.

GUIDED LAB

- **Lab A:** `labs/module-08/l4_3-svm/review_svm_starter.py`. Session 7's reviews, now with a linear SVM: rank with `decision_function`, TF-IDF as the text "scaler", C by cross-validation, the strongest words, the street, calibration.
- **Lab B (pairs):** `labs/module-08/l4_3-svm/kernel_by_hand_starter.py`. Build the degree-2 lift yourself (66 columns), then let a polynomial kernel do it without building anything.
- Done when your numbers match the "Expected when done" block. Finished early? `debug_svm.py`.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code.

P1

Predict the output, then run it. (Four points on a line, two classes.)

```
from sklearn.svm import SVC
m = SVC(kernel="linear", C=1).fit([[0], [1], [3], [4]], [0, 0, 1, 1])
print(m.support_vectors_.ravel(), m.predict([[1.8], [2.2]]))
```

P2

Predict the output, then run it. (Similarity = exp(-gamma × squared distance).)

```
import numpy as np
distance_squared = 4
for gamma in [0.1, 1, 10]:
    print(gamma, round(float(np.exp(-gamma * distance_squared)), 4))
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

A tuned SVM retention list (lesson page, "Project: A Tuned SVM Retention List"). In class: Steps 1 to 3. At home: Steps 4 to 6.

- ☐ Step 1: split keeps the IDs. Check: `4500 1500 ['C102112', 'C103063', 'C101159']`
- ☐ Step 2: logistic regression CV AUC 0.829 ± 0.021
- ☐ Step 3: grid over `svc__C` and `svc__gamma`; best C=10, gamma=0.01, CV AUC 0.854
- ☐ Step 4: test once: SVM 0.848, logistic regression 0.823
- ☐ Step 5: churners in each top 50 (34 and 33) and the overlap (21)
- ☐ Step 6: `svm_shortlist.csv` (scores are distances, not probabilities)

SELF-CHECK

Q1

On overlapping classes you lower a linear SVM's C from 1 to 0.01. What do you expect?

A) A narrower street and fewer support vectors

B) A wider street held up by many more support vectors

C) No change: C only matters for the RBF kernel

D) A perfect training score, because violations are now free

Q2

What does this print?

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
pipe = make_pipeline(StandardScaler(), SVC())
print([key for key in pipe.get_params() if key.endswith("gamma")])
```

Q3

Why does an RBF `SVC` on the churn table need a `StandardScaler` in front of it?

A) The SVM cannot read integer columns

B) Scaling makes the SVM train much faster, nothing else changes

C) Without scaling the model cannot output probabilities

D) The kernel and the margin are built on distances, and unscaled `spend_12m` and `days_since_last_order` would drown out the 0-to-1 columns

Q4

What happens when this runs?

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
pipe = make_pipeline(StandardScaler(), SVC()).fit([[0], [1], [5], [6]], [0, 0,
print(pipe.predict_proba([[3]]))
```

A) It prints `[[0.5 0.5]]`: the point is halfway

B) `AttributeError: This 'Pipeline' has no attribute 'predict_proba'`

C) It prints `[1]`

D) `NotFittedError`: the pipeline was not fitted

Q5

On churn (16.5% churners) you add `class_weight="balanced"` to an RBF SVM. In the lesson, what happened?

A) Recall of churners jumped (0.258 to 0.774) while accuracy fell (0.859 to 0.792)

B) Nothing: class weights only affect logistic regression

C) Accuracy rose and recall fell

D) The model started predicting churn for every customer

Q6

What does this print? (RBF similarity of two points at squared distance 4, for two values of gamma.)

```
import numpy as np
print(round(float(np.exp(-0.1 * 4)), 3), round(float(np.exp(-10 * 4)), 3))
```

JOURNAL

Explain the widest street, support vectors, C and gamma to a colleague who has never seen an SVM, without formulas. Then look at your project's grid table and describe which corner overfit and how you could tell from the numbers alone.

5.1 Decision Trees

BY THE END YOU CAN

- Explain how a classification tree chooses each split by reducing Gini impurity, and compute the impurity of a node by hand
- Train `DecisionTreeClassifier` and `DecisionTreeRegressor`, and read their rules with `export_text` and `plot_tree`
- Explain where a tree's probabilities come from and why trees do not need scaled features
- Control overfitting with `max_depth`, `min_samples_leaf`, `max_leaf_nodes` and cost-complexity pruning, chosen by cross-validation
- Read a tree's feature importances, including what they cannot tell you

KEY IDEAS

- **Signposts, trails, campsites.** Each node asks one yes-or-no question about one feature; a new row walks down to a leaf and gets the leaf's prediction.
- **Gini impurity** = $1 - \Sigma\, p^2$: the chance two rows picked at random from a node disagree. 0 is pure, 0.5 is a 50/50 two-class node. Each split is the (feature, threshold) with the largest weighted decrease, tried for every midpoint.
- **Greedy.** The best split now, never reconsidered; the search repeats inside each child.
- **Probabilities are leaf shares.** Few leaves give coarse ties; pure leaves give 0s and 1s.
- **No scaling needed;** thresholds, U-shapes and interactions come naturally; boxes, not lines.
- **Unlimited trees overfit.** Control size with `max_depth`, `min_samples_leaf`, `max_leaf_nodes` or `ccp_alpha`, chosen by cross-validation.
- **Importances** describe the model's splits, favor many-valued columns, and say nothing about causes. A zero means "the model did not need it".
- **Unstable:** small data changes can change the whole tree (Session 9 turns this into a strength).

WORKED EXAMPLE: A TREE FINDS THE NIGHT-TIME FRAUD

Fraud at night: does the hour matter for every payment, or only for some? A depth-2 tree on two columns answers it.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text

tx = pd.read_csv("data/transactions.csv")
pair = ["hour", "device_new"]
X_train, X_test, y_train, y_test = train_test_split(
    tx[pair], tx["is_fraud"], test_size=0.3, stratify=tx["is_fraud"], random_state
night_tree = DecisionTreeClassifier(max_depth=2, random_state=42).fit(X_train, y_t
print(export_text(night_tree, feature_names=pair, show_weights=True))
```

```
|--- device_new <= 0.50
|   |--- hour <= 15.50
|   |   |--- weights: [16418.00, 73.00] class: 0
|   |--- hour >  15.50
|   |   |--- weights: [15174.00, 86.00] class: 0
|--- device_new >  0.50
|   |--- hour <= 5.50
|   |   |--- weights: [111.00, 82.00] class: 0
|   |--- hour >  5.50
|   |   |--- weights: [2990.00, 66.00] class: 0
```

Read the weights as [legitimate, fraud]. For payments from a known device the tree's second question is about the afternoon, and both leaves are about 0.5% fraud. For a NEW device it asks about the night (hour 5 or earlier): 82 frauds among 193 payments, against 66 among 3,056 during the day. The question about the hour only matters on one side of the tree: that is an interaction.

```
check = X_test.assign(fraud=y_test, night=X_test["hour"] <= 5)
print(check.groupby(["device_new", "night"])["fraud"].mean().round(4).to_string())
```

```
device_new  night
0           False    0.0056
            True     0.0066
1           False    0.0175
            True     0.4079
```

The test payments confirm it: at night from a new device, 40.8% fraud; at night from a known device, 0.66%, no different from the day. A straight-line model with one weight for `hour` cannot say this; a tree says it with two questions.

GUIDED LAB

- **Lab A:** `labs/module-08/l5_1-decision-trees/fraud_rules_starter.py`. A rule book of at most eight rules for the payments team, judged by average precision.

- **Lab B (pairs):** `labs/module-08/l5_1-decision-trees/sales_tree_starter.py`. A regression tree for daily store sales, trained on 2023 and 2024, tested on 2025.
- Done when your numbers match the "Expected when done" block. Finished early? `debug_tree.py`.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code.

P3

Predict the output, then run it. (Gini = 1 - sum of squared shares.)

```
import numpy as np
def gini(counts):
    p = np.asarray(counts) / np.sum(counts)
    return round(float(1 - np.sum(p ** 2)), 3)
print(gini([50, 50]), gini([90, 10]), gini([100, 0]))
```

P4

Predict the output, then run it. (A stump: one question.)

```
from sklearn.tree import DecisionTreeClassifier
X, y = [[1], [2], [3], [10], [11], [12]], [0, 0, 1, 1, 1, 1]
t = DecisionTreeClassifier(max_depth=1, random_state=42).fit(X, y)
print(t.tree_.threshold[0], t.predict_proba([[2.4], [11]]).round(2).tolist())
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

A churn rule book (lesson page, "Project: A Churn Rule Book"). In class: Steps 1 to 3. At home: Steps 4 to 6.

- ☐ Step 1: fifteen columns, no `cancellation_reason`. Check: `(4500, 15) (1500, 15)`
- ☐ Step 2: `tree_prep`: 35 columns after encoding
- ☐ Step 3: baselines: do-nothing 0.500, logistic regression 0.832

- ☐ Step 4: `max_leaf_nodes` by CV; 12 leaves (CV 0.834)
- ☐ Step 5: test once: 0.837
- ☐ Step 6: `churn_rules.txt` with the rules and the segment table

SELF-CHECK

Q7

What does this print? (A node with 30 stayers and 10 churners.)

```
import numpy as np
counts = np.array([30, 10])
p = counts / counts.sum()
print(round(float(1 - np.sum(p ** 2)), 3))
```

Q8

Where does a decision tree's predicted churn probability for a new customer come from?

A) A weighted sum of the features, squashed into 0 to 1

B) The distance from the new customer to the boundary

C) The share of churners among the training customers in the leaf the new customer lands in

D) The average of many trees' votes

Q9

What does this print?

```
from sklearn.tree import DecisionTreeClassifier
t = DecisionTreeClassifier(max_depth=1, random_state=42).fit([[1], [2], [8], [
print(t.tree_.threshold[0])
```

A) `2.0`

B) `5.0`

C) `8.0`

D) `6.5`

Q10

An unlimited churn tree scores 1.000 on the training set and 0.822 on the test set, below the 0.835 baseline. What is the best next move?

A) Report 1.000: the tree learned the data perfectly

B) Scale the features, then refit

C) Switch the criterion from gini to entropy

D) Choose a size control such as `min_samples_leaf` by cross-validation on the training set

Q11

In the pruned churn tree, `spend_12m` gets an importance of 0.012, yet churn falls from 24% to 8% across spend quartiles. What is the right sentence?

A) The tree did not need spend: other features it split on first carried the same information

B) Spend has no relationship with churn

C) Raising a customer's spend would not change their churn risk

D) The importance is wrong because the features were not scaled

Q12

What happens when this runs?

```
from sklearn.tree import DecisionTreeClassifier
DecisionTreeClassifier(max_depth=0).fit([[1], [2]], [0, 1])
```

A) A tree with no questions that predicts the majority class

B) A stump: one question

C) `InvalidParameterError`: max_depth must be at least 1 (or None)

D) An unlimited tree, because 0 means no limit

◆ JOURNAL

Explain the trail-junction signposts to someone who has never seen a decision tree, using one real rule from your rule book. Then describe in two or three sentences why an unlimited tree scores perfectly on training data and badly on test data. Which control would you reach for first, and why?

EXIT TICKET

Three questions, no notes.

E1

Your SVM scores 0.996 on the training customers and AUC 0.699 on the test set, with 4,491 of 4,500 customers as support vectors. Which dial is set wrong, and how do you choose it properly?

E2

A node holds 40 stayers and 10 churners. What is its Gini impurity, and what churn probability does a tree give a customer who lands there?

E3

The care team wants rules they can pin to the wall AND the best possible top-50 list. What do you give them from today's two models, and why not one model for both?

NOTES

HOMEWORK CHECKLIST

- ☐ Finish the SVM retention list (Steps 4 to 6) and the churn rule book (Steps 4 to 6)
- ☐ Lesson 4.3 Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Lesson 5.1 Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Both Quick Quizzes; both journal entries
- ☐ 4.3 "Before the Next Lesson": predict, then check, the CV AUC with gamma=0.003
- ☐ 5.1 "Before the Next Lesson": rerun the instability block with min_samples_leaf=100; write a prediction for the bootstrap question
- ☐ Any unfinished lab from today
- ☐ Bring to Session 9: a READY laptop, `pinecrest-ml` with `data/` and `labs/`, my 5.1 and 4.3 project

notebooks, this workbook and my journal

GLOSSARY

TERM	MEANING
margin (street)	The band around an SVM's boundary; the SVM makes it as wide as it can
support vectors	The training points on or inside the margin (or misclassified); the only ones that shape the boundary
C	The price of a margin violation; small C = wide street, simpler model
kernel	A similarity function that lets an SVM work as if the data were lifted into more features
RBF kernel, gamma	Similarity exp(-gamma × squared distance); gamma sets how far each point reaches
decision_function	An SVM's signed distance from the boundary: a ranking score, not a probability
calibration	Learning a mapping from scores to probabilities (`CalibratedClassifierCV`)
node, root, leaf	A point in a tree; the first node; a node that predicts instead of asking
Gini impurity	$1 - \Sigma\ p^2$: how mixed a node is (0 pure, 0.5 a 50/50 split of two classes)
greedy	Choosing the best split now and never reconsidering it
max_depth / min_samples_leaf / max_leaf_nodes	Limits on questions in a row / the smallest leaf / the number of leaves
cost-complexity pruning (ccp_alpha)	Grow fully, then cut branches that do not earn their keep
impurity importance (MDI)	A feature's total impurity decrease in the fitted tree, normalized to sum to 1

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 9: Random Forests and Gradient Boosting

Your workbook for Session 9: a warm-up on Session 8, the key ideas of Lessons 5.2 and 5.3, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 5.2–5.3 | Session 9 · 3 hours | Name: _____

SESSION GOALS

- Explain the bootstrap and why a bootstrap sample holds about 63% of the rows.
- Explain why averaging many unstable trees works, and why a forest makes its trees less alike.
- Validate a forest out-of-bag, and check its importances with permutation importance.
- Write the boosting loop in words (and in code), and choose the number of trees with early stopping on a validation set.
- Run scikit-learn's, XGBoost's and LightGBM's boosters with native categories, and know which error each ensemble fixes.

WARM-UP: SUPPORT VECTOR MACHINES AND DECISION TREES

Five questions on Session 8. Two minutes alone, then compare with a neighbor.

W1

Lesson 4.3: you raise a linear SVM's C from 0.01 to 100 on overlapping data. Street width and number of support vectors?

W2

An RBF SVM with gamma=1000 scores 1.000 on training data and 0.650 on test data. What happened, and what would have warned you without the test set?

W3

A tree node holds 40 stayers and 10 churners. Gini impurity? Predict, then check.

```
import numpy as np
p = np.array([40, 10]) / 50
print(round(float(1 - np.sum(p ** 2)), 2))
```

W4

Why does an SVM need StandardScaler while a decision tree does not?

W5

Homework check: a bootstrap sample of 4,500 rows drawn with replacement from 4,500. What share of the distinct customers does it contain?

LESSONS

5.2 Bagging and Random Forests

BY THE END YOU CAN

- Draw a bootstrap sample and explain why it contains about 63% of the original rows
- Build a bagged ensemble by hand and with `BaggingClassifier(estimator=...)`, and explain why averaging reduces variance
- Explain how a random forest decorrelates its trees with `max_features`, and tune `n_estimators`, `max_features` and `min_samples_leaf`
- Use out-of-bag predictions as a built-in validation estimate
- Compare impurity-based and permutation importance, and recognize the bias of the first

KEY IDEAS

- **A panel of new hires.** Many quirky models whose quirks point different ways: average them and the quirks cancel.
- **The bootstrap** draws n rows from n with replacement: about 63.2% of the distinct rows; the other 36.8% are out-of-bag.
- **Variance of an average** = $\rho\sigma^2 + (1 - \rho)\sigma^2/n$. More models shrink one term; less alike models shrink the other.
- **Bagging** helps unstable models (deep trees) and cannot fix bias. `BaggingClassifier(estimator=...)`.
- **A random forest** = bagged trees + a random subset of features at every split (`max_features`): weaker, less alike trees, a better average.
- **More trees never overfit;** the score plateaus. Leaf size matters more; forests are forgiving to tune.
- **Out-of-bag** predictions give a validation score from one fit (not after target-based preprocessing, not for time-ordered rows).
- **Importances:** MDI rewards many-valued columns and chance splits; check with permutation importance on held-out data. Both describe the model.

WORKED EXAMPLE: HOW SURE IS A CHURN RATE?

The bootstrap is not only for forests: it tells you how sure a number is. How precise is the churn rate of Pinecrest's 475 Gold customers?

```
import numpy as np
import pandas as pd

customers = pd.read_csv("data/customers_churn.csv")
gold = customers.loc[customers["loyalty_tier"] == "Gold", "churned"].to_numpy()
print(len(gold), "Gold customers; churn rate", round(gold.mean(), 4))
```

```
475 Gold customers; churn rate 0.0653
```

Resample the 475 customers with replacement 2,000 times and recompute the rate each time:

```
rng = np.random.default_rng(42)
rates = [gold[rng.integers(0, len(gold), size=len(gold))].mean() for _ in range(20
low, high = np.percentile(rates, [2.5, 97.5])
print(f"bootstrap: 95% of resampled rates between {low:.3f} and {high:.3f}")
```

```
bootstrap: 95% of resampled rates between 0.044 and 0.088
```

Each resample holds about 63% of the distinct Gold customers, some twice or more, exactly like one tree's sample in a forest. The spread of the 2,000 rates says the true Gold churn rate could plausibly be anywhere in that interval: with only 475 customers, "6.5%" deserves a plus-or-minus. Lesson 5.4 uses the same trick to ask whether one model's lead over another is real.

GUIDED LAB

- **Lab A:** `labs/module-09/l5_2-random-forests/fraud_forest_starter.py`. A forest for the fraud review queue, tuned out-of-bag, tested once against logistic regression and one tree.
- **Lab B (pairs):** `labs/module-09/l5_2-random-forests/decorrelate_starter.py`. Measure the variance formula as max_features moves.
- Done when your numbers match the "Expected when done" block. Finished early? `debug_forest.py`.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code.

P1

Predict the output, then run it. (Four trees' 0/1 votes for three customers.)

```
import numpy as np
votes = np.array([[1, 0, 1],      # tree 1's churn votes for three customers
                  [1, 1, 0],      # tree 2
                  [0, 0, 1],      # tree 3
                  [1, 0, 1]])     # tree 4
print(votes.mean(axis=0))
```

P2

Predict the output, then run it. (The variance of an average.)

```
rho, var = 0.5, 1.0
for n in [1, 10, 1000]:
    print(n, round(rho * var + (1 - rho) * var / n, 3))
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

A churn forest and a call list (lesson page, "Project: A Churn Forest and a Call List"). In class: Steps 1 and 2. At home: Steps 3 to 6.

- ☐ Step 1: the 5.1 columns and split. Check: `(4500, 15) (1500, 15) C102112`
- ☐ Step 2: leaf size by OOB AUC (best 0.868 at 10)
- ☐ Step 3: one test, four models (forest 0.863)
- ☐ Step 4: real churners in each top 50 (forest 41)
- ☐ Step 5: permutation importance on the original columns
- ☐ Step 6: `call_list.csv`, 50 rows, no outcome column

SELF-CHECK

Q1

What does this print? (The share of distinct rows in a bootstrap sample of n rows from n.)

```
n = 1000
print(round(1 - (1 - 1 / n) ** n, 3))
```

Q2

Which change makes the trees of a random forest LESS correlated with each other?

A) Raising `n_estimators`

B) Setting `max_features=None`

C) Lowering `max_features`

D) Raising `min_samples_leaf` to 500

Q3

For one training customer, which trees produce the out-of-bag prediction?

A) Only the trees whose bootstrap sample did not include that customer

B) All trees in the forest

C) Trees refitted on a held-out fold

D) The trees that predicted that customer correctly

Q4

What happens when this runs?

```
from sklearn.ensemble import RandomForestClassifier
RandomForestClassifier(bootstrap=False, oob_score=True).fit([[0], [1], [2], [3
```

A) It fits, and oob_score_ is 1.0

B) It fits, and oob_score_ is missing

C) A warning that too few trees were used

D) `ValueError: Out of bag estimation only available if bootstrap=True`

Q5

What does this print? (The variance of an average of n models with correlation 0.3.)

```
rho, var = 0.3, 1.0
print([round(rho * var + (1 - rho) * var / n, 3) for n in (1, 10, 100)])
```

Q6

A column of pure random noise gets an impurity importance (MDI) of 0.065 in a forest, but a permutation importance of 0.005 on the test set. Best conclusion?

A) The column carries real signal that permutation importance missed

B) MDI was inflated: a many-valued column wins chance splits on the training data

C) The column has a small causal effect on churn

D) The forest needs more trees

◆ JOURNAL

Explain to a colleague, with the panel of new hires, why 100 overfitted trees can predict better than one carefully pruned tree. Use the variance formula's two terms. Then write down your project's numbers: the

forest's test AUC, the rule tree's, and the two top-50 counts. When would you still hand the care team the rule tree?

5.3 Gradient Boosting

BY THE END YOU CAN

- Explain how boosting adds small trees one after another, each fitted to the remaining errors, and how the learning rate controls the step size
- Train `HistGradientBoostingClassifier` and `HistGradientBoostingRegressor` on data with missing values and native categorical columns
- Use early stopping on a validation set to choose the number of trees without touching the test set
- Run the same model with `XGBClassifier` and `LGBMClassifier`, including their categorical and early-stopping options
- Choose between a random forest, scikit-learn's boosting and the external libraries for a given problem

KEY IDEAS

- **Putting on a golf green.** Start from the average; each small tree aims at what is still wrong; add a fraction of it: F = F + v · tree.
- **Gradient:** for squared error the residual is the negative gradient; for churn each tree fits y − p on the log-odds scale.
- **Training error falls forever; validation error turns around.** Early stopping on a validation set picks the number of trees.
- **Learning rate:** smaller steps need more trees and usually generalize a little better. Recipe: small rate, high max_iter, early stopping, then tree size.
- **HistGradientBoosting:** binned features, missing values handled, `category` columns split natively; early stopping "auto" only above 10,000 rows.
- **XGBoost and LightGBM:** the same estimator API; you pass the validation set (`eval_set`; `eval_X` / `eval_y`), never the test set.
- **Choose the loss for the target:** Poisson for counts, quantile for "enough stock".
- **Forest vs boosting:** forests reduce variance and are hard to break; boosting reduces bias and usually wins once tuned.

WORKED EXAMPLE: BOOSTING DAILY STORE SALES

Session 8's Lab B fitted one regression tree to daily store sales. A booster on the same calendar information, with the store and the event as native categories:

```
import pandas as pd
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.metrics import mean_absolute_error

sales = pd.read_csv("data/daily_store_sales.csv", parse_dates=["date"])
X = pd.DataFrame({"store": sales["store"].astype("category"),
                  "day_of_week": sales["date"].dt.dayofweek, "month": sales["date"
                  "promo": sales["promo"], "event": sales["event"].fillna("none").
is_train = sales["date"].dt.year < 2025
booster = HistGradientBoostingRegressor(random_state=42).fit(X[is_train], sales.lo
pred = booster.predict(X[~is_train])
print(f"test MAE ${mean_absolute_error(sales.loc[~is_train, 'sales'], pred):,.0f}
```

```
test MAE $969   trees 100
```

```
check = pd.DataFrame({"store": sales.loc[~is_train, "store"], "actual": sales.loc[
print(check.groupby("store")[["actual", "predicted"]].mean().round(0))
```

```
           actual  predicted
store
Asheville  4380.0     4625.0
Bend       5527.0     5195.0
Boise      8342.0     7332.0
Flagstaff  6561.0     5355.0
```

About the same as Session 8's depth-10 tree ($976): with only calendar information there is little left for more trees to find. And look at the store table: Boise and Flagstaff grew in 2025 and are under-predicted, Asheville shrank and is over-predicted. A boosted model is still a sum of trees, and every leaf value comes from 2023 and 2024: it cannot extrapolate a trend either. More trees do not fix a missing piece of information.

GUIDED LAB

- **Lab A:** `labs/module-09/l5_3-gradient-boosting/boosting_by_hand_starter.py`. Write the boosting loop for churn and compare it with the library.
- **Lab B (pairs):** `labs/module-09/l5_3-gradient-boosting/fraud_boosters_starter.py`. Three libraries on fraud, early-stopped on validation; then the peeking version.
- Done when your numbers match the "Expected when done" block. Finished early? `debug_boosting.py`.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code.

P3

Predict the output, then run it. (Each step covers half of what is left.)

```
import numpy as np
pred, truth = np.array([10.0]), np.array([14.0])
for step in range(3):
    pred = pred + 0.5 * (truth - pred)
    print(pred)
```

P4

Predict the output, then run it. (The starting guess for a 20% churn rate.)

```
import numpy as np
rate = 0.2
start = np.log(rate / (1 - rate))
print(round(float(start), 3), round(float(1 / (1 + np.exp(-start))), 3))
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

A weekly demand forecaster (lesson page, "Project: Weekly Demand Forecaster"). In class: Steps 1 and 2. At home: Steps 3 to 6.

- ☐ Step 1: split by year. Check: 5,088 training rows, 4,896 test
- ☐ Step 2: baselines 17.13 / 13.42 / 13.81
- ☐ Step 3: HistGradientBoosting with native categories, MAE 8.16
- ☐ Step 4: Poisson loss, MAE 8.05, no negative forecasts
- ☐ Step 5: MAE per category; one product's season
- ☐ Step 6: `demand_forecast_2025.csv`

SELF-CHECK

Q7

What does this print? (Current predictions 20 and 20, truths 26 and 14, learning rate 0.1, a tree that fits the residuals perfectly.)

```
import numpy as np
pred = np.array([20.0, 20.0])
truth = np.array([26.0, 14.0])
print(pred + 0.1 * (truth - pred))
```

Q8

Bagging and boosting both combine trees. Which statement is right?

A) Bagging averages independent deep trees and mainly reduces variance; boosting adds small trees in sequence, each fixing what is still wrong, and mainly reduces bias

B) Both grow deep trees on bootstrap samples; boosting just uses more of them

C) Boosting averages the trees; bagging adds them

D) Bagging needs early stopping; boosting never overfits

Q9

You fit `HistGradientBoostingClassifier()` with all defaults on the 4,500 training customers. How many trees does it build?

A) It stops at the best number of trees on an internal validation set

B) 2,000

C) 100: early_stopping="auto" is off below 10,000 rows, so it builds max_iter=100

D) As many as there are features

Q10

What happens when this runs?

```
import pandas as pd
from sklearn.ensemble import HistGradientBoostingClassifier
X = pd.DataFrame({"tenure": [3, 40, 5, 60], "store": ["Bend", "Boise", "Bend",
HistGradientBoostingClassifier().fit(X, [1, 0, 1, 0])
```

A) It fits, treating store as categorical

B) `ValueError: could not convert string to float: 'Bend'`

C) It fits, one-hot encoding store automatically

D) A warning, then store is dropped

Q11

You early-stop an XGBClassifier. What goes into eval_set?

A) The test set, so it stops where it will be graded

B) The training rows it is fitted on

C) Nothing: XGBoost holds out its own validation set

D) A validation set split off from the training rows

Q12

You forecast weekly units sold (counts, never negative) with HistGradientBoostingRegressor and get 45 negative forecasts. What is the best first fix?

A) Clip the forecasts at zero and move on

B) Use loss="poisson", which predicts on a log scale

C) Scale the target with StandardScaler

D) Add more trees

Explain the difference between a random forest and gradient boosting to a Pinecrest store manager in three sentences, without the words "variance" or "gradient". Then write the two numbers from today that would convince that manager the forecaster is worth using, and one assumption they should challenge.

EXIT TICKET

Three questions, no notes.

E1

A forest and a boosted model are both made of trees. In two sentences: how are their trees different, and which kind of error does each mainly fix?

E2

Why is a forest's out-of-bag score free, and when should you not trust it?

E3

A colleague early-stops XGBoost with eval_set=[(X_test, y_test)] and reports AUC 0.867. What is wrong, and what do you tell them?

NOTES

HOMEWORK CHECKLIST

- ☐ Finish the churn forest and call list (Steps 3 to 6) and the demand forecaster (Steps 3 to 6)
- ☐ Lesson 5.2 Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Lesson 5.3 Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Both Quick Quizzes; both journal entries
- ☐ 5.2 "Before the Next Lesson": refit the forest with 50 and 1,000 trees
- ☐ 5.3 "Before the Next Lesson": random_state=7 rerun; write my prediction about averaging logistic regression with boosting
- ☐ A first idea for my final project: churn, demand or fraud
- ☐ Bring to Session 10: a READY laptop, `pinecrest-ml` with `data/` and `labs/`, my 5.2 and 5.3 notebooks, my written prediction, this workbook and my journal

GLOSSARY

TERM	MEANING
bootstrap sample	n rows drawn with replacement from n rows; about 63.2% distinct
out-of-bag (OOB)	The rows a tree never saw; their trees' average gives a free validation prediction
bagging	Fit one model per bootstrap sample, average the predictions
random forest	Bagged trees with a random subset of features tried at each split (`max_features`)
decorrelation	Making the models less alike (smaller ρ) so their average cancels more error
permutation importance	How much a score drops on held-out data when one column is shuffled
boosting	Adding small models one after another, each fitted to what is still wrong
learning rate (ν)	The fraction of each new tree added to the model
early stopping	Stopping when the validation score has not improved for a set number of rounds
residual / negative gradient	What the current model still gets wrong; what the next tree is fitted to
log-odds	log(p / (1 − p)): the scale on which a boosted classifier adds its trees
Poisson loss	A loss for counts: positive predictions, multiplicative effects

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 10: Ensembles and Model Selection

Your workbook for Session 10: the key ideas of Lessons 5.4 and 6.1, new worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, your final-project choice and plan, and your exit ticket.

Lessons 5.4-6.1 | Session 10 · 3 hours | Name: _____

SESSION GOALS

- Explain when a team of models beats its best member, and measure how different their mistakes are.
- Build soft votes and stacks from out-of-fold predictions, and recognize the silent in-sample leak.
- Compare models fairly: one splitter, one metric chosen first, paired per-fold differences, the test set opened once.
- Read validation and learning curves and say what would help next: features, regularization or more data.
- Choose your final project and write a plan you can defend.

LESSONS

5.4 Voting and Stacking

BY THE END YOU CAN

- Explain why a group of models beats its members only when their errors differ, and measure that diversity
- Build hard and soft `VotingClassifier` ensembles from complete preprocessing pipelines
- Build a stack by hand from out-of-fold predictions, then with `StackingClassifier`, and explain why in-sample predictions leak
- Judge whether an ensemble's gain is real (bootstrap) and worth its cost in training time, latency and explainability

KEY IDEAS

- **A crowd is wise only when its members disagree.** Independent 70% voters reach 0.837 in fives; when most copy one opinion, adding voters barely helps.
- **Measure diversity on out-of-fold predictions** (`cross_val_predict`), never on the test set. Forest and booster correlate 0.89 on churn: near-copies.
- **Soft voting** averages probabilities and keeps `predict_proba`; **hard voting** counts labels and loses it. Choose weights out-of-fold.
- **Stacking** fits a simple meta-model on out-of-fold member predictions; `StackingClassifier(..., cv=cv)` does it in one estimator. Its weights describe the combination, not the customers.
- **In-sample meta-features leak:** the meta-model over-trusts the overconfident member and can score below its own best member, with no warning.
- **Blending** uses one holdout instead of folds: simpler, noisier, natural for time-ordered data (blend on a later period).
- **Ship an ensemble only when the gain survives a bootstrap or CV and changes a decision**, and is worth 19 fits and four pieces to run instead of one.

WORKED EXAMPLE: A STRONG AND DIFFERENT MEMBER

Lesson 5.4's logistic regression was the weak, different member. What if it were strong AND different? Give it the shapes a straight line cannot draw (Lesson 2.3): a flag for customers quiet for more than 150 days, flags for very new and very old customers, and support tickets counted only for customers without the app.

```
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import StratifiedKFold, cross_val_predict, train_test
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

customers = pd.read_csv("data/customers_churn.csv")
numeric = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "days_si
           "returns_12m", "return_rate", "support_tickets_12m", "discount_share",
categorical = ["home_store", "region", "age_group", "loyalty_tier", "preferred_cat

def add_shapes(df):                        # the shapes a straight line cannot draw
    return df.assign(lapsed=(df["days_since_last_order"] > 150).astype(int),
                     new_cust=(df["tenure_months"] < 6).astype(int),
                     veteran=(df["tenure_months"] > 72).astype(int),
                     tickets_no_app=df["support_tickets_12m"] * (1 - df["app_user"

X = add_shapes(customers[numeric + categorical])
y = customers["churned"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, stratify
shapes = ["lapsed", "new_cust", "veteran", "tickets_no_app"]
prep = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median"), StandardScaler()), num
    ("cat", make_pipeline(SimpleImputer(strategy="most_frequent"), OneHotEncoder(h
logreg_shapes = make_pipeline(prep, LogisticRegression(max_iter=1000))
boost = make_pipeline(ColumnTransformer([("cat", OneHotEncoder(handle_unknown="ign
                                         remainder="passthrough"), HistGradientBoos

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
oof = pd.DataFrame({name: cross_val_predict(m, X_train, y_train, cv=cv, method="pr
                    for name, m in [("logreg_shapes", logreg_shapes), ("boost", bo
print("out-of-fold ROC AUC:", {c: round(roc_auc_score(y_train, oof[c]), 3) for c i
print("correlation:", round(oof.corr().iloc[0, 1], 3))
print("out-of-fold ROC AUC of the average:", round(roc_auc_score(y_train, oof.mean
```

```
out-of-fold ROC AUC: {'logreg_shapes': 0.877, 'boost': 0.862}
correlation: 0.878
out-of-fold ROC AUC of the average: 0.877
```

On its own the feature-built logistic regression (0.877 out-of-fold) beats the booster (0.862), and the two still disagree a fair amount (correlation 0.878). Yet their average adds nothing out-of-fold. Check it once on the test customers:

```
test = pd.DataFrame({name: m.fit(X_train, y_train).predict_proba(X_test)[:, 1]
                     for name, m in [("logreg_shapes", logreg_shapes), ("boost", b
for c in ["logreg_shapes", "boost"]:
    print(f"{c:14s} test ROC AUC {roc_auc_score(y_test, test[c]):.3f}")
print(f"{'soft vote':14s} test ROC AUC {roc_auc_score(y_test, test.mean(axis=1)):.
```

```
logreg_shapes  test ROC AUC 0.874
boost          test ROC AUC 0.866
soft vote      test ROC AUC 0.876
```

A gain of 0.002 over the best member. The big step came from the features, not the team: once one member knows the shapes, the other has little left to add. Ensembles polish; features transform.

GUIDED LAB

- **Lab A (pairs):** `labs/module-10/l5_4-voting-stacking/fraud_ensemble_starter.py`. Card fraud, trained on January to May, judged on June. Choose the team on out-of-fold predictions, build a stack, open June once, and bootstrap the gain. One types, one predicts; swap at TODO 3.
- Done when your numbers match the "Expected when done" lines.
- Finished early? `debug_stack.py` (three loud bugs, one silent), then `demand_blend_starter.py` (blending on a later period: the time-aware way).

PREDICT, THEN RUN

Write your prediction on the line first, then run the code.

P1

Predict the output, then run it. (Three models vote 0 or 1 for three customers.)

```
import numpy as np
votes = np.array([[1, 0, 1],
                  [0, 0, 1],
                  [1, 1, 1]])          # three models, three customers
print((votes.sum(axis=1) >= 2).astype(int))
```

P2

Predict the output, then run it. (A second model that only rescales the first.)

```
import numpy as np
a = np.array([0.10, 0.40, 0.80, 0.30])
b = 2 * a + 0.05                        # a "second model" that only rescales th
print(round(np.corrcoef(a, b)[0, 1], 3), np.argsort(a).tolist() == np.argsort(
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Should Pinecrest Ship the Ensemble? (lesson page). In class: Steps 1 to 3. At home: Steps 4 and 5.

- ☐ Step 1: `(4500, 16) (1500, 16) ['logreg', 'forest', 'boost']`
- ☐ Step 2: five contenders; stack ROC AUC 0.864, boost 0.862
- ☐ Step 3: churners in the top 150: logreg 87, forest 96, boost 98, soft vote 96, stack 98 (random 25)
- ☐ Step 4: paired bootstrap; is the interval clear of zero?
- ☐ Step 5: timings and a recommendation printed from my variables

SELF-CHECK

Q1

Three churn models have out-of-fold probabilities that correlate 0.98 with each other, and each scores about 0.86 ROC AUC. What should you expect from a soft vote of the three?

A) A large gain, because three models always beat one

B) About 0.86: members that make the same mistakes add almost nothing

C) A score below 0.5, because correlated members cancel out

D) An error: a soft vote needs uncorrelated members

Q2

What does this print?

```
import numpy as np
probs = np.array([[0.30, 0.80, 0.60],
                  [0.10, 0.40, 0.45]])       # 2 customers x 3 models
print(probs.mean(axis=1).round(2), (probs > 0.5).sum(axis=1))
```

Q3

A stack's meta-model was fitted on each member's predictions for its own training rows. Its weights come out as logreg -3.25, forest 2.69, boost 12.66. What went wrong?

A) Nothing: the booster is simply the best member

B) The meta-model needs scaled inputs, so the weights are meaningless

C) The members were not tuned before stacking

D) In-sample predictions are overconfident, so the meta-model over-trusts the most flexible member

Q4

`hard_vote.predict_proba(X_test)` raises `AttributeError`. You need a ranked call list. What is the fix?

A) Build the vote with `voting="soft"`, from members that all have `predict_proba`

B) Call `hard_vote.predict(X_test)` and sort by it

C) Add `probability=True` to the VotingClassifier

D) Use a StackingClassifier with a hard vote inside it

Q5

What happens when this runs?

```
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
vote = VotingClassifier([LogisticRegression(), DecisionTreeClassifier()], voti
vote.fit([[0], [1], [2], [3]], [0, 0, 1, 1])
```

A) It fits a soft vote of the two models

B) It fits, but only the first model is used

C) `ValueError`: every member needs a name, as a (string, estimator) pair

D) `TypeError`: a VotingClassifier needs at least three members

Q6

A paired bootstrap of the stack minus the single booster gives a mean gain of +0.0023 ROC AUC, 95% interval -0.0070 to +0.0124. Both put 98 real churners in their top 150. What is the professional recommendation?

A) Ship the stack: its mean gain is positive

B) Ship the stack only if it is retrained every week

C) Rerun the bootstrap on the training rows until the interval excludes zero

D) Keep the single booster: the gain is within the noise and changes no call, and the stack has 19 fits and four pieces to run

◆ JOURNAL

Think of a decision you make by weighing several opinions. How do you learn whose advice to trust, and how is that like a meta-model trained on out-of-fold predictions? Then write your project's recommendation in one sentence, and the single number that would change your mind.

6.1 Model Selection and Learning Curves

BY THE END YOU CAN

- Compare several models fairly with `cross_validate`: the same preprocessing pipeline, the same folds and the same metric, reporting the spread and the fit time
- Decide whether a gap between two models is bigger than the noise, using paired per-fold differences
- Draw a validation curve with `ValidationCurveDisplay` and point to where a model moves from underfitting to overfitting
- Draw a learning curve with `LearningCurveDisplay` and say whether more data or a more flexible model is the better next step
- Make a final choice you can defend, then score it once on the untouched test set

KEY IDEAS

- **Testing tents:** same campsites (folds), same nights (repeats), compare night by night. One splitter object with a fixed `random_state` for every candidate.
- `cross_validate` returns per-fold scores and fit times, for several metrics at once. Choose the deciding metric before looking.
- **Paired differences** cancel shared fold difficulty. Worth acting on: the winner wins most folds and the mean difference is clearly larger than its std / √(number of folds). Otherwise: within noise.
- **Validation curve** (one knob): both scores low = underfitting; training near perfect and validation falling = overfitting. Pipeline knobs are `step__param`.
- **Learning curve** (more data): curves that meet and flatten = high bias (better features or a more flexible model); a wide gap still closing = variance (regularize or add data).
- **Within noise? Prefer the simpler model** (the one-standard-error rule).
- **Open the test set once, after the choice,** and report its score as is.

WORKED EXAMPLE: FEATURES AGAINST A STRONGER MODEL, FOLD BY FOLD

Is the feature-built logistic regression really better than default boosting at the top of the list, or did it get one lucky split? Score both on the same 10 folds with average precision and compare fold by fold. (Run it after the 5.4 worked example, in the same notebook.)

```
from sklearn.model_selection import RepeatedStratifiedKFold, cross_val_score

rcv = RepeatedStratifiedKFold(n_splits=5, n_repeats=2, random_state=42)
scores = {name: cross_val_score(m, X_train, y_train, cv=rcv, scoring="average_prec
          for name, m in [("logreg_shapes", logreg_shapes), ("boost", boost)]}
for name, s in scores.items():
    print(f"{name:14s} CV AP {s.mean():.3f} +/- {s.std():.3f}")
diff = scores["logreg_shapes"] - scores["boost"]
print(f"logreg_shapes - boost: mean {diff.mean():+.3f}, std {diff.std():.3f}, wins
```

```
logreg_shapes  CV AP 0.640 +/- 0.028
boost          CV AP 0.596 +/- 0.030
logreg_shapes - boost: mean +0.045, std 0.015, wins 10 of 10
```

A lead of 0.045 in every one of the 10 folds, with the differences varying by only 0.015: a real difference, not luck. A simple model with good features beat a flexible model without them, which is what a flat learning curve for logistic regression predicted.

GUIDED LAB

- **Lab A:** `labs/module-10/l6_1-model-selection/fraud_bakeoff_starter.py`. Four fraud models on the same 10 folds of January to May, average precision chosen in advance, paired differences, June opened once.
- Before TODO 1 runs, predict: which model will the AUC column favor, and which the AP column?
- Finished early? `debug_bakeoff.py`, then `demand_curves_starter.py` (shuffled folds vs blocks of weeks for a forecaster).

PREDICT, THEN RUN

Write your prediction on the line first, then run the code.

P3

Predict the output, then run it. (How many train/validation splits?)

```
from sklearn.model_selection import RepeatedStratifiedKFold
print(RepeatedStratifiedKFold(n_splits=3, n_repeats=4, random_state=42).get_n_
```

P4

Predict the output, then run it. (How does a pipeline name its tree's depth?)

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
pipe = make_pipeline(StandardScaler(), DecisionTreeClassifier())
print([name for name in pipe.get_params() if name.endswith("max_depth")])
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The Churn-Model Bake-off (lesson page). In class: Steps 1 to 3. At home: Steps 4 to 6.

- ☐ Step 1: `4500 1500`
- ☐ Step 2: five candidate pipelines
- ☐ Step 3: leaderboard on 10 shared folds; forest AP 0.615, baseline 0.165
- ☐ Step 4: paired check of the top two
- ☐ Step 5: learning curve for the winner, one sentence on more data
- ☐ Step 6: test once; `bakeoff_results.csv`

SELF-CHECK

Q7

Why does the lesson pass ONE splitter object, created with a fixed random_state, to every candidate's cross_validate call?

A) So the test set takes part in the comparison

B) So every candidate is scored on exactly the same folds and the scores can be compared fold by fold

C) So each model is fitted only once

D) Because cross_validate refuses an integer cv

Q8

What does this print?

```
import numpy as np
a = np.array([0.84, 0.87, 0.88, 0.85, 0.86])   # model A, 5 shared folds
b = np.array([0.83, 0.85, 0.89, 0.84, 0.84])   # model B, same folds
diff = a - b
print(round(diff.mean(), 3), (diff > 0).sum())
```

Q9

A validation curve for a tree shows training AUC 0.857 and validation AUC 0.835 at depth 4, and training AUC 1.000 with validation AUC 0.673 at depth 15. What is happening at depth 15?

A) Underfitting: the tree is too simple

B) Nothing: a perfect training score means the model is right

C) Overfitting: the tree memorizes the training customers

D) The validation folds contain an error

Q10

Logistic regression's learning curve flattens at about 0.83 by 1,200 rows, with training and validation scores close together. Which next step is most likely to help?

A) Better features or a more flexible model

B) Collecting twice as many rows of the same data

C) Stronger regularization

D) A larger test set

Q11

You call `validation_curve(tree_pipe, ..., param_name="max_depth", ...)` where tree_pipe was built with make_pipeline. What happens?

A) It works: the pipeline passes max_depth to the tree

B) `ValueError`: the pipeline has no max_depth; use `decisiontreeclassifier__max_depth`

C) It silently varies the scaler instead

D) `KeyError`: max_depth

Q12

One-standard-error rule. What does this print?

```
import numpy as np
depths = [3, 4, 5, 6]
means = np.array([0.829, 0.835, 0.834, 0.829])
se = np.array([0.007, 0.007, 0.007, 0.008])
best = int(np.argmax(means))
print(next(d for d, m in zip(depths, means) if m >= means[best] - se[best]))
```

JOURNAL

Explain the tent test in your own words: what are the campsites, the nights and the leaks? Then sketch the two learning curves from the lesson and write one sentence under each saying what you would try next and why.

FINAL PROJECT: MY CHOICE AND PLAN

One supervised model, built end to end: frame, leak-safe split, baseline, pipelines, a fair comparison, tuning, one look at the test set, a decision in business terms, and an honest one-page report. Choose one problem; Session 11 is the studio; submit one week after Session 11. The rubric is in your instructor's final-assessment brief.

PROBLEM	DATA AND SPLIT	DECISION AND METRIC	BAR TO BEAT
A · Churn	`customers_churn.csv`; stratified 75/25, `random_state=42`	Who gets a retention offer ($15 per offer, $60 per churner reached)	Contact everyone: -$7,620; random-order AP 0.165
B · Weekly demand	`weekly_demand.csv`; train 2024, test 2025	Units per product, store and week (MAE in units)	Per-product mean MAE 13.42
C · Fraud	`transactions.csv`; train January to May, test June	Which payments to review ($10 per review, $375 per missed fraud)	Review nothing: $24,000

My problem (circle): **A · Churn** / **B · Weekly demand** / **C · Fraud**

1

The decision my model informs, and who makes it:

2

The target, and the moment the prediction is made (what is not yet known then):

3

The split (copy it from the problem card):

4

My deciding metric, chosen now, and why:

5

The bar to beat (what doing nothing scores):

6

Two candidate models I will compare on the same folds:

7

One leakage risk specific to my problem, and how I avoid it:

8

The result that would change my recommendation:

Now write it as one paragraph in the first Markdown cell of `final_project.ipynb`. Milestone 1 before Session 11: run `labs/module-11/l6_2-hyperparameter-tuning/final_project_starter.py` with `PROBLEM` set to your choice.

EXIT TICKET

E1

Why must a stack's meta-model learn from out-of-fold predictions?

E2

Two models: CV AUC 0.867 and 0.860; paired differences mean +0.007, std 0.008, the first wins 9 of 10 folds. Is the gap real? What else decides?

E3

Your final project: which problem, which deciding metric, and what does doing nothing score?

NOTES

HOMEWORK CHECKLIST

- ☐ 5.4 project Steps 4 and 5; 6.1 project Steps 4 to 6
- ☐ Lesson 5.4 Exercises 1 to 3; Lesson 6.1 Exercises 1 to 3 (Exercise 4 of each is a stretch)
- ☐ Both Quick Quizzes; both journal entries
- ☐ 6.1 "Before the Next Lesson": three boosting hyperparameter names; a guess for closing boosting's gap
- ☐ Final project: plan in `final_project.ipynb`; milestone 1 with `final_project_starter.py`
- ☐ Bring to Session 11: laptop, `pinecrest-ml` with `data/` and `labs/`, my 6.1 project notebook, my final-project notebook, this workbook

GLOSSARY

TERM	MEANING
ensemble	A model made of several models whose predictions are combined
soft / hard voting	Averaging probabilities / taking the majority of 0-1 labels
out-of-fold (OOF) prediction	A prediction for a training row from a model fitted on the other folds
stacking	A meta-model trained on the members' out-of-fold predictions
meta-model (final estimator)	The second-level model in a stack; keep it simple
blending	Stacking with a single holdout (for time data: a later period)
paired bootstrap	Resampling the same test rows for two models to see the spread of their difference
paired difference	Score of model A minus model B on the same fold
validation curve	Training and validation scores as one hyperparameter changes
learning curve	Training and validation scores as the amount of training data grows
high bias / high variance	Underfitting (too simple) / overfitting (memorizing)
one-standard-error rule	Choose the simplest setting within one standard error of the best

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 11: Hyperparameter Tuning and Final Project Studio

Your workbook for Session 11: Lesson 6.2's key ideas, two new worked examples, the guided lab, the project-skills self-check, your final-project studio page and showcase card, and your exit ticket.

Lesson 6.2 + studio | Session 11 · 3 hours | Name: _____

SESSION GOALS

- Tune a whole pipeline with grid search, random search, successive halving and Optuna, naming its knobs as `step__param`.
- Spend a search budget well: log scales, distributions, and a written space before anything runs.
- Report a tuned model honestly: best_score_ chooses, a nested estimate or one test look reports.
- Move your final project through milestones 2 and 3 in the studio, and present it in two minutes.

LESSON

6.2 Hyperparameter Tuning

BY THE END YOU CAN

- Tune a whole pipeline with `GridSearchCV` and `RandomizedSearchCV`, naming its hyperparameters as `step__param`, and read `cv_results_`
- Choose sensible search ranges (log scales, distributions from `scipy.stats`) and explain why random search usually beats a grid of the same size
- Speed up a search with successive halving (`HalvingRandomSearchCV`)
- Run a seeded Optuna study that learns where to search next
- Explain why `best_score_` is optimistic, estimate a tuned model honestly with nested cross-validation, and use the test set once

KEY IDEAS

- **Parameters** are learned by `fit`; **hyperparameters** are chosen by you, before `fit`, and found by search with cross-validation on the training rows.
- **Every search** has a space (knobs and ranges), a budget (trials) and an objective (the CV metric decided first). Keys are `step__param`.
- **Grid search** tries every combination: its cost is the product of the list lengths. **Random search** samples distributions (`loguniform`, `randint`) and usually beats a grid of the same size.
- **Successive halving** screens many candidates on growing data; slow starters can be knocked out. **Optuna** (TPE) learns from past trials; seed it and run trials one at a time.
- **best_score_ is optimistic** (the winner's curse). Report a nested cross-validation estimate or the test score, once. On churn: best 0.8795, nested 0.868, test 0.874.
- **Tuning moves a model toward its own ceiling**; it does not change which family has the higher one. When gains flatten, look at features and framing.

WORKED EXAMPLE: A BUSINESS THRESHOLD, TUNED HONESTLY

Tuning a model is one decision; turning probabilities into actions is another, and it must be tuned honestly too. Take the tuned churn booster from Lesson 6.2's project and let TunedThresholdClassifierCV choose the offer threshold on the training customers for Lesson 1.4's economics ($15 per offer, $60 per churner reached).

```
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.impute import SimpleImputer
from sklearn.metrics import make_scorer
from sklearn.model_selection import StratifiedKFold, TunedThresholdClassifierCV, t
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

customers = pd.read_csv("data/customers_churn.csv")
numeric = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "days_si
           "returns_12m", "return_rate", "support_tickets_12m", "discount_share",
categorical = ["home_store", "region", "age_group", "loyalty_tier", "preferred_cat
train, test = train_test_split(customers, test_size=0.25, stratify=customers["chur
X_train, y_train = train[numeric + categorical], train["churned"]
X_test, y_test = test[numeric + categorical], test["churned"]
prep = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median"), StandardScaler()), num
    ("cat", make_pipeline(SimpleImputer(strategy="most_frequent"), OneHotEncoder(h
best = {"l2_regularization": 0.0292, "learning_rate": 0.0472, "max_leaf_nodes": 50
tuned = Pipeline([("prep", prep), ("model", HistGradientBoostingClassifier(random_

def campaign_profit(y_true, flagged):              # Lesson 1.4: $60 per churner
    y_true, flagged = np.asarray(y_true), np.asarray(flagged)
    return 60 * int(((flagged == 1) & (y_true == 1)).sum()) - 15 * int(flagged.sum

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
plan = TunedThresholdClassifierCV(tuned, scoring=make_scorer(campaign_profit), cv=
offers = plan.predict(X_test)
print(f"threshold chosen on the training customers: {plan.best_threshold_:.3f}  (b
print(f"test: {offers.sum()} offers, {int(y_test[offers == 1].sum())} reach a chur
```

```
threshold chosen on the training customers: 0.240  (break-even 0.25)
test: 377 offers, 182 reach a churner, profit $5,265
```

Now the tempting shortcut: try thresholds on the test customers and keep the best.

```
p = plan.predict_proba(X_test)[:, 1]
peek = {t: campaign_profit(y_test, (p >= t).astype(int)) for t in np.round(np.aran
t_peek = max(peek, key=peek.get)
print(f"threshold picked by peeking at the test set: {t_peek:.2f}, 'profit' ${peek
print(f"default 0.5: ${campaign_profit(y_test, (p >= 0.5).astype(int)):,}")
print(f"contact everyone: {campaign_profit(y_test, np.ones(len(p), int)):,} dollar
```

```
threshold picked by peeking at the test set: 0.26, 'profit' $5,475
default 0.5: $3,690
contact everyone: -7,620 dollars
```

The chosen threshold (0.240) sits next to the break-even 0.25, a sign the probabilities are reasonably calibrated, and earns $5,265 on customers it never saw. Peeking finds $5,475, but that number was chosen by the data that grades it, so it is not a result: it is the winner's curse again. The default 0.5 earns far less, and offering to everyone loses money.

◆ WORKED EXAMPLE: WHEN TUNING DOES NOT HELP

Tuning does not always help, and saying so is a result. The demand forecaster from Lesson 5.3, tuned with 8 random settings on blocks of 2024 weeks:

```
import pandas as pd
from scipy.stats import loguniform, randint
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import KFold, RandomizedSearchCV, cross_val_score

demand = pd.read_csv("data/weekly_demand.csv")
features = ["store", "product_id", "category", "week_of_year", "list_price",
            "discount_pct", "price", "promo", "holiday_week", "avg_high_temp_f"]
cats = ["store", "product_id", "category"]
X = demand[features].copy()
X[cats] = X[cats].astype("category")
is_train = demand["year"] == 2024
X_train, X_test = X[is_train], X[~is_train]
y_train, y_test = demand.loc[is_train, "units_sold"], demand.loc[~is_train, "units

blocks = KFold(5)                                        # blocks of weeks: no shuf
base = HistGradientBoostingRegressor(loss="poisson", random_state=42)
default_cv = -cross_val_score(base, X_train, y_train, cv=blocks, scoring="neg_mean
space = {"learning_rate": loguniform(0.01, 0.3), "max_leaf_nodes": randint(4, 64),
         "min_samples_leaf": randint(10, 200), "l2_regularization": loguniform(1e-
search = RandomizedSearchCV(base, space, n_iter=8, cv=blocks, scoring="neg_mean_ab
                            random_state=42, n_jobs=-1).fit(X_train, y_train)
print(f"default CV MAE {default_cv:.2f}   best of 8 settings {-search.best_score_:
print(f"2025 test MAE: default {mean_absolute_error(y_test, base.fit(X_train, y_tr
      f"   tuned {mean_absolute_error(y_test, search.predict(X_test)):.2f}")
```

```
default CV MAE 8.49   best of 8 settings 8.73
2025 test MAE: default 8.05   tuned 8.20
```

The best of 8 settings is worse than the library default in cross-validation (8.73 against 8.49), and the test agrees (8.20 against 8.05). The honest report: "A random search did not improve on the default settings; we kept the default." Then spend the effort where it pays: features, the forecast horizon, or more history.

GUIDED LAB

- **Lab A:** `labs/module-11/l6_2-hyperparameter-tuning/tune_fraud_starter.py`. Write the space, run a budgeted search on January to May, read the leaderboard, open June once for both models, save the settings.
- Done when your numbers match the "Expected when done" lines.
- Finished early? `winners_curse_starter.py`: tune a tree on shuffled labels and watch best_score_ climb on pure noise.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code.

P1

Predict the output, then run it. (Settings and fits in a grid.)

```
from sklearn.model_selection import ParameterGrid
grid = {"model__learning_rate": [0.05, 0.1], "model__max_leaf_nodes": [7, 15,
        "model__l2_regularization": [0.0]}
print(len(ParameterGrid(grid)), len(ParameterGrid(grid)) * 5)
```

P2

Predict the output, then run it. (How many rows does each demand year have?)

```
import pandas as pd
demand = pd.read_csv("data/weekly_demand.csv")
print((demand["year"] == 2024).sum(), (demand["year"] == 2025).sum())
```

P3

Predict the output, then run it. (Two break-even probabilities.)

```
print(round(15 / 60, 2), round(10 / 375, 3))
```

LESSON PROJECT: AT HOME

Tune the Retention Model (lesson page). This one goes home: the studio belongs to your final project.

- ☐ Step 1: default CV average precision 0.608 +/- 0.043
- ☐ Step 2: the space and the budget (15 settings x 5 folds = 75 fits)
- ☐ Step 3: best CV AP 0.636
- ☐ Step 4: settings within one std of the best: 12 of 15
- ☐ Step 5: test AP 0.600 → 0.614; top-50 shortlist 40 → 43 real churners
- ☐ Step 6: `best_params.json`

SELF-CHECK: PROJECT SKILLS

Q1

Which of these is a well-framed final-project question?

A) Build the most accurate churn model possible

B) Use gradient boosting on the transactions table

C) Which customers should get the $15 retention offer next quarter, judged by profit on held-out customers, against offering to everyone (-$7,620)?

D) Find out what causes churn

Q2

Your weekly-demand model (train 2024, test 2025) has an MAE of 9.1 units. Which number must it beat to be worth anything?

A) The per-product 2024 mean: MAE 13.42

B) Zero, the perfect forecast

C) The training MAE of the same model

D) A shuffled 75/25 split's MAE of 7.36

Q3

A learner adds "units sold last week" as a feature to the demand forecaster, which is meant to plan all of 2025 at the start of the year. What is wrong?

A) Nothing: lag features always help

B) Lag features need scaling first

C) The feature makes the model slower

D) On the planning date, last week's sales are unknown for almost every 2025 week: it is a framing leak

Q4

How many fits does a GridSearchCV with cv=5 run on this grid, not counting the refit?

```
from sklearn.model_selection import ParameterGrid
grid = {"model__learning_rate": [0.03, 0.1, 0.3],
        "model__max_leaf_nodes": [7, 15, 31, 63]}
print(len(ParameterGrid(grid)) * 5)          # cv=5
```

Q5

Your search reports best_score_ 0.636 (CV average precision). The tuned model scores 0.614 on the test set, opened once. Which number goes in your report as the model's expected performance?

A) 0.636, because it is the cross-validated score

B) 0.614, the one-time test score (or a nested CV estimate), with best_score_ described only as what chose the settings

C) Their average, 0.625

D) Whichever is higher, since both are honest

Q6

Two hyperparameters, only the first matters, budget 9 trials. Why does random search usually beat a 3 × 3 grid?

A) Random search uses more folds

B) Random search learns from earlier trials

C) It tries 9 different values of the important hyperparameter instead of 3

D) A grid cannot use log scales

Q7

For the fraud project, which split is the leak-safe one?

A) Train January to May, test June, never shuffling months together

B) A stratified 70/30 split of all six months, random_state=42

C) Train on June, test on January to May

D) Five-fold cross-validation on all six months, with no test set

Q8

What does this print?

```
import numpy as np
print(np.logspace(-3, 1, 5))
```

Q9

A churn offer costs $15 and is worth $60 when it reaches a real churner. If the probabilities are well calibrated, above which churn probability does an offer pay for itself?

A) 0.50, the default of predict

B) 0.75

C) 0.15

D) 0.25

Q10

Which sentence is an honest report of a tuning result?

A) Tuning made the model 2% better

B) Tuned boosting: test ROC AUC 0.874 against 0.860 for the default, on the same 1,500 test customers, after a search on the training rows only; the nested CV estimate of the procedure is 0.868

C) Our best CV score was 0.8795, so the model will score 0.88 on new customers

D) Tuning proved that small trees cause better predictions

Q11

You choose the churn offer threshold by trying thresholds on the TEST customers and keeping the most profitable (0.26: $5,475). The threshold chosen by TunedThresholdClassifierCV on the training customers (0.240) earns $5,265 on the same test set. Which number goes in the report?

A) $5,475: it is the best result you found

B) The average of the two

C) $5,265: the threshold was chosen without the test set, so its test profit is an honest estimate

D) Neither: profits cannot be reported

Q12

Fraud review cost ($10 per review, $375 per missed fraud). What does this print?

```
from sklearn.metrics import confusion_matrix
y_true = [0, 0, 0, 1, 1, 0, 1, 0]
flag   = [1, 0, 0, 1, 0, 1, 1, 0]
tn, fp, fn, tp = confusion_matrix(y_true, flag).ravel()
print(10 * (tp + fp) + 375 * fn)
```

JOURNAL

Write down three numbers from today: the default model's cross-validated score, the best tuned score, and the nested or test score. Explain to an imagined manager, in two or three sentences, why the middle number is not the one to promise. Then describe the camp stove analogy for one of the four search strategies.

FINAL PROJECT STUDIO

THE MILESTONE LADDER (TICK WHAT IS DONE)

- ☐ **1 · Split and bar to beat:** the course's split for my problem; the do-nothing numbers; a first CV score (`final_project_starter.py`)
- ☐ **2 · Fair leaderboard:** baseline + at least two real candidates as pipelines, ONE splitter, my deciding metric, mean ± std, a paired check of the top two

- ☐ **3 · Tuned on the training rows:** a written space (log scales), 10 to 15 settings; churn and fraud: a threshold tuned for the business cost on the training rows
- ☐ **4 · Test once, decide:** fit the chosen model on all training rows, open the test set once, compute the business number (profit, MAE, cost)
- ☐ **5 · Report:** one page (Question, Answer, Evidence, Caveats, Next questions, Method), README, a notebook that reruns from a fresh start

MY LEADERBOARD (TRAINING ROWS ONLY)

CANDIDATE	MEAN (DECIDING METRIC)	STD	NOTES
baseline			

STUDIO QUESTIONS TO ASK MYSELF

- Which rows chose each setting? (Training rows only.)
- On prediction day, which of my columns would I not know yet?
- Is the gap between my top two bigger than the fold spread?
- What will I report: best_score_, a nested estimate, or the one-time test score?

SHOWCASE: MY TWO MINUTES

S1

The decision my model informs:

S2

The bar to beat (what doing nothing scores):

S3

My best cross-validated score, with its spread:

S4

One trap I avoided, and how:

S5

What is left before I submit:

Peer feedback I received (one strength, one question):

EXIT TICKET

E1

Why is a search's `best_score_` not the number to promise?

E2

Write the three numbers your report will lead with.

E3

What is the one thing left before you submit?

NOTES

HOMEWORK CHECKLIST

- ☐ Lesson 6.2 project Steps 1 to 6 and `best_params.json`
- ☐ Lesson 6.2 Exercises 1 to 3 (Exercise 4 is a stretch); the Quick Quiz; the journal
- ☐ Final project milestones 4 and 5; submit within one week

- ☐ Optional: `winners_curse_starter.py`

GLOSSARY

TERM	MEANING
hyperparameter	A setting chosen before fit (learning_rate, max_leaf_nodes, C)
search space	The knobs and ranges a search may try
grid / random search	Every combination of listed values / samples from distributions
loguniform	A distribution even across orders of magnitude, for rates and strengths
successive halving	A tournament: many candidates on little data, the best go on with more
Optuna, TPE	A tuning library whose sampler learns from earlier trials
best_score_	The best mean CV score of a search: chooses, but is optimistic
winner's curse	The best of many noisy scores looks better than it really is
nested cross-validation	Cross-validating the whole search-and-refit procedure
decision threshold	The probability above which you act; tuned for costs on training data
milestone	A checkpoint of the final project: split, leaderboard, tuning, test once, report

Complete Participant Workbook · all 11 sessions · v1.0 · © 2026 Ray de la Paz